

Massachusetts Institute of Technology

MIT Taxi

Jacob Teo, Siyong Huang, Thomas Guo

2026-11-09

Contest (1)

template.cpp

16 lines

```cpp
#include <bits/stdc++.h>
using namespace std;

#define rep(i,a,b) for(int i = (a); i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define mp make_pair
#define pb push_back
using ll = long long;
using pii = pair<int,int>;
using vi = vector<int>;

bool ckmax(auto &a, auto const& b) {return b>a?a=b,1:0;}
bool ckmin(auto &a, auto const& b) {return b<a?a=b,1:0;}

int main() { cin.tie(0);cin.sync_with_stdio(0); }
```

.bashrc

5 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++14 \
  -fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps =
    <>
setxkbmap -option caps:escape # bind caps to esc
setxkbmap -option # remove options
```

.vimrc

17 lines

```
set ts=2 sw=2 ai cin nu rnu udf udir=~/.vim/udir
sy on
set cul ru nowrap wmnu sc is bs=indent,eol,start cino=g0
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \
 \| md5sum \| cut -c-6

nmap <F8> :w <Bar> !g++ -std=c++20 -DLOCAL %<CR>
nmap <F9> :w <Bar> !g++ -std=c++20 -DLOCAL % && ./a.out<CR>

autocmd FileType python set sw=4 ts=4 sts=4 et nocin si
autocmd FileType python nmap <F8> :w <Bar> !python3 "%"<CR>
autocmd FileType python nmap <F9> :w <Bar> !python3 -i "%"<
    CR>

" To map caps lock -> escape --
" setxkbmap -option caps:escape
```

hash.sh

3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use
    for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -
    c-6
```

troubleshoot.txt

52 lines

```
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it
    .

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

Mathematics (2)

2.1 Equations

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i'th column replaced by b.

2.2 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

2.3 Trigonometry

$$(V + W)\tan(v - w)/2 = (V - W)\tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$a\cos x + b\sin x = r\cos(x - \phi)$$
$$a\sin x + b\cos x = r\sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}$, $\phi = \operatorname{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

For side lengths a, b, c, and $p = \frac{a+b+c}{2}$, $A = \sqrt{p(p-a)(p-b)(p-c)}$

Circumradius: $R = \frac{abc}{4A}$, Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

2.4.2 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 Pick's Theorem

Polygon with integer vertices: $A = i + \frac{b}{2} - 1$

2.5 Derivatives/Integrals

$$\frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx}\tan x = 1 + \tan^2 x \qquad \frac{d}{dx}\arctan x = \frac{1}{1+x^2}$$
$$\int \tan ax = -\frac{\ln|\cos ax|}{a} \qquad \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2}$$
$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) \qquad \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1)$$

2.6 Sums

$$\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}; \sum_{i=1}^{n} i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \le 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \le x \le 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

$$\frac{1}{1-x} = 1 + x + x^2 + \ldots = \sum_{n\ge 0} x^n$$

$$-\ln(1-x) = x + \frac{x^2}{2} + \frac{x^3}{3} + \ldots = \sum_{n\ge 1}\frac{x^n}{n}$$

$$(1+x)^r = \sum_{n\ge 0}\binom{r}{n}x^n$$

2.8 Probability theory

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\text{Bin}(n,p)$, $n = 1, 2, \ldots$, $0 \le p \le 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}; \mu = np,\ \sigma^2 = np(1-p)$$

$\text{Bin}(n,p)$ is approximately $\text{Po}(np)$ for small p.

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each wich yields success with probability p is $\text{Fs}(p)$, $0 \le p \le 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots; \mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\text{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda}\frac{\lambda^k}{k!}, k = 0, 1, 2, \ldots; \mu = \lambda,\ \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \ge 0 \\ 0 & x < 0 \end{cases};\ \mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

2.9 Markov chains

Transition matrix: $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$ π is a stationary distribution if $\pi = \pi\mathbf{P}$. If *irreducible* (any state to any state possible): $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i. π_j/π_i is the expected number of visits in state j between two visits in state i. For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i's degree. A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k\to\infty} \mathbf{P}^k = \mathbf{1}\pi$. A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j, is $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik}p_{kj}$. The expected time until absorption, when the initial state is i, is $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki}t_k$.

2.10 Graphs

2.10.1 Erdos-Gallai theorem

A simple graph with node degrees $d_1 \ge \cdots \ge d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \le k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$$

2.10.2 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--, mat[b][b]++` (and `mat[b][a]--, mat[a][a]++` if G is undirected). Remove the ith row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

Data structures (3)

FastStaticRMQ.h

Description: Static RMQ min(V[a], V[a + 1], ... V[b]) in constant time.
Usage: `RMQ rmq(values); rmq.query(inclusive, inclusive);`
Time: $\mathcal{O}(N + Q)$

e0058a, 28 lines

```
template<typename T> struct RMQ {
  vector<T> v;
  int n; static const int b = 30;
  vector<int> mask, t;

  int op(int x, int y) { return v[x] < v[y] ? x : y; }
  int msb(int x) { return __builtin_clz(1)-__builtin_clz(x); }
  int small(int r, int sz = b) { return r-msb(mask[r]&((1<<sz)-1)); }
  RMQ(const vector<T>& v_) : v(v_), n(v.size()), mask(n), t(n) {
    for (int i = 0, at = 0; i < n; mask[i++] = at |= 1) {
      at = (at<<1)&((1<<b)-1);
      while (at and op(i, i-msb(at&-at)) == i) at ^= at&-at;
    }
    for (int i = 0; i < n/b; i++) t[i] = small(b*i+b-1);
    for (int j = 1; (1<<j) <= n/b; j++) for (int i = 0; i+(1<<j) <= n/b; i++)
      t[n/b*j+i] = op(t[n/b*(j-1)+i], t[n/b*(j-1)+i+(1<<(j-1))]);
  }
  T query(int l, int r) {
    if (r-l+1 <= b) return v[small(r, r-l+1)];
    int ans = op(small(l+b-1), small(r));
    int x = l/b+1, y = r/b-1;
    if (x <= y) {
      int j = msb(y-x+1);
      ans = op(ans, op(t[n/b*j+x], t[n/b*j+y-(1<<j)+1]));
    }
    return v[ans];
  }
};
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.
Time: Both operations are $\mathcal{O}(\log N)$.

e62fac, 22 lines

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
  }
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos-1];
    return res;
  }
  int lower_bound(ll sum) {// min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    }
    return pos;
  }
};
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).
Time: $\mathcal{O}\left(\log^2 N\right)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

"FenwickTree.h" 157f07, 22 lines

```
struct FT2 {
  vector<vi> ys; vector<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
  }
  void init() {
    for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
  }
  int ind(int x, int y) {
    return (int)(lower_bound(all(ys[x]), y) - ys[x].begin());
  }
  void update(int x, int y, ll dif) {
    for (; x < sz(ys); x |= x + 1)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x-1].query(ind(x-1, y));
    return sum;
  }
};
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d77092, 7 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

LazySegmentTree.h

Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.
Usage: `Node* tr = new Node(v, 0, sz(v));`
Time: $\mathcal{O}(\log N)$.

"../various/BumpAllocator.h" 34ecf5, 50 lines

```
const int inf = 1e9;
struct Node {
  Node *l = 0, *r = 0;
  int lo, hi, mset = inf, madd = 0, val = -inf;
  Node(int lo,int hi):lo(lo),hi(hi){} // Large interval of -inf
  Node(vi& v, int lo, int hi) : lo(lo), hi(hi) {
    if (lo + 1 < hi) {
      int mid = lo + (hi - lo)/2;
      l = new Node(v, lo, mid); r = new Node(v, mid, hi);
      val = max(l->val, r->val);
    }
    else val = v[lo];
  }
  int query(int L, int R) {
    if (R <= lo || hi <= L) return -inf;
    if (L <= lo && hi <= R) return val;
```

```
    push();
    return max(l->query(L, R), r->query(L, R));
  }
  void set(int L, int R, int x) {
    if (R <= lo || hi <= L) return;
    if (L <= lo && hi <= R) mset = val = x, madd = 0;
    else {
      push(), l->set(L, R, x), r->set(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void add(int L, int R, int x) {
    if (R <= lo || hi <= L) return;
    if (L <= lo && hi <= R) {
      if (mset != inf) mset += x;
      else madd += x;
      val += x;
    }
    else {
      push(), l->add(L, R, x), r->add(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void push() {
    if (!l) {
      int mid = lo + (hi - lo)/2;
      l = new Node(lo, mid); r = new Node(mid, hi);
    }
    if (mset != inf)
      l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
    else if (madd)
      l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
  }
};
```

LineContainer.h

Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").
Time: $\mathcal{O}(\log N)$ 8ec1c7, 30 lines

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y)
      );
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change `step` to add/remove the edge (a, c) and remove the initial `add` call (but keep `in`).
Time: $\mathcal{O}(N\sqrt{Q})$ a12ef4, 49 lines

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or
    1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
  int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;
#define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]);
    });
  for (int qi : s) {
    pii q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
  }
  return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=
    0){
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  };
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk &
    1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]);
    });
  for (int qi : s) rep(end,0,2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  }
  return res;
}
```

PBBST.h

Description: Persistent AVL tree with split. Nodes made since the last `step()` are modified in place, so `split` and `+` destroy their operands unless those are older. `step()` freezes all existing trees and returns a fresh modifiable copy. Nodes are never freed: call `PAVL::reset()` between testcases (invalidates every existing tree).
Usage: `PAVL a = PAVL(Node(1)) + PAVL(Node(2));`
`PAVL b = a.step(); // a is now immutable`
`auto [l, r] = b.split(1); // consumes b, not a`
Time: $\mathcal{O}(\log N)$ 0b33ad, 83 lines

```
struct PAVL {
  struct Node {
    int t;
    int s, h; /* Customize */
    array<int, 2> c;
    int val;
    Node() : t(), s(1), h(1), c{ -1, -1 } { t = T; }
    Node(int _val) : Node() { val = _val; }
    void up() {}; /* Customize */
    void down() {};
  };
  static vector<Node> N;
  static int T;
  static int clone(int n) {
    if (n == -1) return -1; //assert(N[n].t >= t);
    if (N[n].t == T) return n;
    return N.push_back(N[n]), N.back().t = T, N.size() - 1;
  }
  static int gh(int n) { return n != -1 ? N[n].h : 0; }
  static int gs(int n) { return n != -1 ? N[n].s : 0; }
  static void up(int n) {
    N[n].h = max(gh(N[n].c[0]), gh(N[n].c[1])) + 1;
    N[n].s = gs(N[n].c[0]) + gs(N[n].c[1]) + 1;
    N[n].up();
  }
  static int down(int n) { n = clone(n); return N[n].down(),
      n; }
  static int rotate(int n, int d) {
    n = clone(n); int o = down(N[n].c[d]);
    N[n].c[d] = N[o].c[!d], N[o].c[!d] = n;
    up(n), up(o);
    return o;
  }
  static int balance(int n) {
    assert(N[n].t == T); up(n);
    int diff = gh(N[n].c[0]) - gh(N[n].c[1]), d;
    if (diff >= 2) d = 0;
    else if (diff <= -2) d = 1;
    else return n;
    N[n].c[d] = down(N[n].c[d]);
    if (gh(N[N[n].c[d]].c[d]) + 1 < gh(N[n].c[d]))
      N[n].c[d] = rotate(N[n].c[d], !d);
    return rotate(n, d);
  }
  static int merge_root(int l, int n, int r) {
    if (gh(l) + 1 < gh(r))
      return r = down(r), N[r].c[0] = merge_root(l, n, N[r].
     c[0]), balance(r);
    else if (gh(r) + 1 < gh(l))
      return l = down(l), N[l].c[1] = merge_root(N[l].c[1],
     n, r), balance(l);
    else return N[n].c = { l, r }, balance(n);
  }
  static tuple<int, int> split(int n, int k) {
    if (n != -1) n = down(n);
    if (k == 0) return { -1, n };
    if (k == N[n].s) return { n, -1 };
    if (k <= gs(N[n].c[0])) {
      auto [l, r] = split(N[n].c[0], k);
      return { l, merge_root(r, n, N[n].c[1]) };
    } else {
      auto [l, r] = split(N[n].c[1], k - gs(N[n].c[0]) - 1);
      return { merge_root(N[n].c[0], n, l), r };
    }
  }
  static int merge(int l, int r) {
    if (r == -1) return l;
    auto [x, nr] = split(r, 1);
    return merge_root(l, clone(x), nr);
  }
  PAVL(int v) : root(v) {}
  int root;
  PAVL() : root(-1) {}
  PAVL(Node&& n) : root(N.size()) { N.push_back(n); }
  friend PAVL operator+(PAVL a, PAVL b) { return merge(a.
      root, b.root); }
  tuple<PAVL, PAVL> split(int k) {
    auto [l, r] = split(root, k);
    return { PAVL(l), PAVL(r) };
  }
  PAVL step() { ++T; return clone(root); }
  static void reset() { N.clear(); T = 0; }
  Node& get_root() { return N[root]; }
};
typedef PAVL::Node Node;
vector<Node> PAVL::N;
int PAVL::T;
```

PBDS.h

Description: examples for PBDS BBST, mergeable heaps and rope.
Time: $\mathcal{O}(\log N)$
<ext/pb_ds/assoc_container.hpp>, <ext/pb_ds/tree_policy.hpp>, <ext/pb_ds/priority_queue.hpp>, <ext/rope> dcaf87, 37 lines

```
using namespace std;
using namespace __gnu_pbds;
using namespace __gnu_cxx;
template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
template<class T>
using Heap = __gnu_pbds::priority_queue<T, less<T>,
    pairing_heap_tag>;
```

```
//binary_heap_tag, pairing_heap_tag, binomial_heap_tag,
    rc_binomial_heap_tag, thin_heap_tag
void pbds() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).first;
  assert(it == t.lower_bound(9));
  assert(t.order_of_key(10) == 1);
  assert(t.order_of_key(11) == 2);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t

  Heap<int> pq1, pq2;
  pq1.push(1); pq2.push(5);
  pq1.join(pq2); // merge pq2 into pq1
  assert(pq1.top()==5);
  auto pq_it = pq1.push(3);
  assert(pq1.top()==5);
  pq1.modify(pq_it,7); // modify-key in O(log N)
  assert(pq1.top()==7);

  int n=3;
    rope<int> v(n, 0);
    for (int i=0; i<n; i++) v.mutable_reference_at(i) = i + 1; // (1 2 3)
  for (int i=0; i<n; i++) v.push_back(i + n + 1); // (1 2 3 4 5 6)
    int l=1, r=3;
    rope<int> cur = v.substr(l, r-l+1); // 2 3 4
    v.erase(l, r-l+1); // 1 5 6
    v.insert(v.mutable_begin(), cur); //to start (2 3 4 1 5 6)
    // v.insert(1, cur); // to ONE AFTER start (1 2 3 4 5 6)
    // v.insert(v.mutable_begin() + 2, cur); // to TWO AFTER start (1 5 2 3 4 6)
}
```

PersistentSegmentTree.h

Description: Persistent lazy segment tree (range add, range sum) with path copying. All problem-specific code lives in `Node`: `Node()` must be the identity (empty segment), `apply` applies a lazy tag to a segment of length `len`, `up` combines children, `down` pushes the tag to child copies. Bounds are $[lo, hi)$ and passed as arguments; root 0 is the all-identity tree, so `build` is optional and coordinates up to 10^9 work without compression. Mutating ops return the new root. Nodes created since the last `step()` are modified in place (the returned root may equal the input); older versions are never changed. Nodes are never freed: `reset()` between testcases. For point updates only, delete `lz`, `down` and the calls to `down`.

Usage: `int r0 = pst::build(v, 0, n); // or r0 = 0`
`pst::step(); int r1 = pst::add(r0, 0, n, l, r, 5); // r0 intact`
`ll s = pst::query(r1, 0, n, l, r).v;`

Time: $\mathcal{O}(\log N)$ per op; $\leq 4 \log N$ new nodes per range op, $2 \log N$ per point op (half that without lazy).

08b008, 54 lines

```
namespace pst {
struct Node {
  array<int, 2> c = {0, 0}; int t = -1;
  ll v = 0, lz = 0; /* Customize below */
  void apply(ll x, int len) { v += x * len, lz += x; }
  void up(const Node& a, const Node& b) { v = a.v + b.v; }
  void down(Node& a, Node& b, int la, int lb) {
    a.apply(lz, la), b.apply(lz, lb), lz = 0;
  }
};
vector<Node> N(1); int T = 0;
void step() { ++T; }
void reset() { N.assign(1, Node()); T = 0; }
int clone(int x) {
  if (N[x].t == T) return x;
  N.push_back(N[x]); N.back().t = T;
  return sz(N) - 1;
}
void up(int x) { N[x].up(N[N[x].c[0]], N[N[x].c[1]]); }
void down(int x, int lo, int hi) { // x must be cloned
  int m = (lo + hi) / 2;
  int a = clone(N[x].c[0]), b = clone(N[x].c[1]);
  N[x].c = {a, b};
  N[x].down(N[a], N[b], m - lo, hi - m);
}
int build(vector<ll>& v, int lo, int hi) {
  int x = clone(0), m = (lo + hi) / 2;
  if (lo + 1 == hi) return N[x].v = v[lo], x;
  N[x].c = {build(v, lo, m), build(v, m, hi)};
  return up(x), x;
}
int add(int x, int lo, int hi, int l, int r, ll val) {
  if (r <= lo || hi <= l) return x;
  x = clone(x);
  if (l <= lo && hi <= r) return N[x].apply(val, hi - lo), x;
  down(x, lo, hi);
  int m = (lo + hi) / 2;
  N[x].c[0] = add(N[x].c[0], lo, m, l, r, val);
  N[x].c[1] = add(N[x].c[1], m, hi, l, r, val);
  return up(x), x;
}
Node query(Node x, int lo, int hi, int l, int r) {
  if (r <= lo || hi <= l) return Node();
  if (l <= lo && hi <= r) return x;
  int m = (lo + hi) / 2;
  Node a = N[x.c[0]], b = N[x.c[1]], res;
  x.down(a, b, m - lo, hi - m);
  res.up(query(a, lo, m, l, r), query(b, m, hi, l, r));
  return res;
}
Node query(int x, int lo, int hi, int l, int r) {
  return query(N[x], lo, hi, l, r);
}
}
```

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.

Usage: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`

Time: $\mathcal{O}(|V| \log |V| + Q)$

510c32, 16 lines

```
template<class T>
struct RMQ {
  vector<vector<T>> jmp;
  RMQ(const vector<T>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
      jmp.emplace_back(sz(V) - pw * 2 + 1);
      rep(j,0,sz(jmp[k]))
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }
  T query(int a, int b) {
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
  }
};
```

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.

Time: $\mathcal{O}(\log N)$

ed34c6, 21 lines

```
struct Tree {
  typedef int T;
  static constexpr T unit = INT_MIN;
  T f(T a, T b) { return max(a, b); } // (any associative fn)
  vector<T> s; int n;
  Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {
    for (int i = n; --i > 0;) s[i] = f(s[i*2], s[i*2+1]);
  }
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2) ra = f(ra, s[b++]);
      if (e % 2) rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

SparseSegTree2D.h

Description: 2D point-update range-query segtree supporting 10^9 coordinates (IOI 2013 game)

Time: $\mathcal{O}\left(\log^2 N\right)$

40b942, 92 lines

```
#define DEFAULT 0ll
ll func(ll a, ll b) {return max(a, b);} // associative func
struct SegTree2D {
  int R,C,root;
  vector<int> l,r,b,e,st;
  vector<ll> v;
  inline int mid(int x,int y) {return ((x+y)>>1);}
  SegTree2D(int _R,int _C):R(_R),C(_C) {
    l.pb(0),r.pb(0),b.pb(0),e.pb(0),st.pb(0),v.pb(DEFAULT);
    R=_R;
    root=alloc2(0,R-1);
  }
  int alloc(int _b,int _e,ll _v) {
    l.pb(0),r.pb(0),b.pb(_b),e.pb(_e),st.pb(0),v.pb(_v);
    return sz(b)-1;
  }
  void lca(int b,int e,int ob,int oe,int i,int &nb,int &ne) {
    int m=mid(b,e);
    if ((i<=m&&ob>m)||(i>m&&oe<=m)) nb=b,ne=e;
    else (i>m)?lca(m+1,e,ob,oe,i,nb,ne):lca(b,m,ob,oe,i,nb,ne);
  }
  void up(int x) {v[x]=func(v[l[x]],v[r[x]]);}
  void update1(int x,int i,ll nv) {
    if (b[x]>i||e[x]<i) return;
    if (b[x]==e[x]) {
      v[x]=nv;
      return;
    }
    int m=mid(b[x],e[x]);
    if (i<=m) {
      if (l[x]) {
        if (b[l[x]]<=i && i<=e[l[x]]) update1(l[x],i,nv);
        else {
          int nb,ne;
          lca(0,C-1,b[l[x]],e[l[x]],i,nb,ne);
          int y=l[x];
          l[x]=alloc(nb,ne,DEFAULT);
          if (i>mid(nb,ne)) l[l[x]]=y,r[l[x]]=alloc(i,i,nv);
          else r[l[x]]=y,l[l[x]]=alloc(i,i,nv);
          up(l[x]);
        }
      } else l[x]=alloc(i,i,nv);
    } else {
      if (r[x]) {
        if (b[r[x]]<=i && i<=e[r[x]]) update1(r[x],i,nv);
        else {
          int nb,ne;
          lca(0,C-1,b[r[x]],e[r[x]],i,nb,ne);
          int y=r[x];
          r[x]=alloc(nb,ne,DEFAULT);
          if (i>mid(nb,ne)) l[r[x]]=y,r[r[x]]=alloc(i,i,nv);
          else r[r[x]]=y,l[r[x]]=alloc(i,i,nv);
          up(r[x]);
        }
      } else r[x]=alloc(i,i,nv);
    }
    up(x);
  }
  ll query1(int x,int qb,int qe) {
    if (!x) return DEFAULT;
    if (b[x]>qe||e[x]<qb) return DEFAULT;
    if (b[x]>=qb&&e[x]<=qe) return v[x];
    return func(query1(l[x],qb,qe),query1(r[x],qb,qe));
  }
  int alloc2(int _b,int _e) {
    int newnode = alloc(0,C-1,DEFAULT);
    l.pb(0),r.pb(0),b.pb(_b),e.pb(_e),v.pb(DEFAULT),st.pb(newnode);
    return sz(b)-1;
  }
  void update2(int x,int i,int j,ll nv) {
    if (b[x]>i||e[x]<i) return;
    if (b[x]==e[x]) update1(st[x],j,nv);
    else {
      int m=mid(b[x],e[x]);
      if (!l[x]) {
        l[x]=alloc2(b[x],m);
        r[x]=alloc2(m+1,e[x]);
      }
```

```
      if (i<=m) update2(l[x],i,j,nv);
      else update2(r[x],i,j,nv);
      update1(st[x],j,func(query1(st[l[x]],j,j),query1(st[r[x]],j,j)));
    }
  }
  ll query2(int x,int rb,int re,int cb,int ce) {
    if (!x) return DEFAULT;
    if (b[x]>re||e[x]<rb) return DEFAULT;
    if (b[x]>=rb&&e[x]<=re) return query1(st[x],cb,ce);
    return func(query2(l[x],rb,re,cb,ce),query2(r[x],rb,re,cb,ce));
  }
  void update(int p,int q,ll k) {update2(root,p,q,k);}
  ll query(int p,int q,int u,int v) {return query2(root,p,u,q,v);}
};
```

Treap-benq.h

Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data. 0-index.
Time: $\mathcal{O}(\log N)$ c8a465, 72 lines

```
using pt = struct Node*;
struct Node {
  int pri, val; pt c[2]; // essential
  int sz; ll sum; // for range queries
  bool flip = 0; // lazy update
  Node(int _val) {
    pri = rand(); sum = val = _val;
    sz = 1; c[0] = c[1] = nullptr;
  }
  ~Node() { rep(i,0,2) delete c[i]; }
};
int getsz(pt x) { return x?x->sz:0; }
ll getsum(pt x) { return x?x->sum:0; }
pt prop(pt x) { // lazy propagation
  if (!x || !x->flip) return x;
  swap(x->c[0],x->c[1]);
  x->flip = 0; rep(i,0,2) if (x->c[i]) x->c[i]->flip ^= 1;
  return x;
}
pt calc(pt x) {
  pt a = x->c[0], b = x->c[1];
  assert(!x->flip); prop(a), prop(b);
  x->sz = 1+getsz(a)+getsz(b);
  x->sum = x->val+getsum(a)+getsum(b);
  return x;
}
void tour(pt x, vi& v) { // print values of nodes,
  if (!x) return; // inorder traversal
  prop(x); tour(x->c[0],v); v.pb(x->val); tour(x->c[1],v);
}
pair<pt,pt> split(pt t, int v) { // >= v goes to the right
  if (!t) return {t,t};
  prop(t);
  if (t->val >= v) {
    auto p = split(t->c[0], v); t->c[0] = p.second;
    return {p.first,calc(t)};
  } else {
    auto p = split(t->c[1], v); t->c[1] = p.first;
    return {calc(t),p.second};
  }
}
pair<pt,pt> splitsz(pt t, int sz) { // sz nodes go to left
  if (!t) return {t,t};
  prop(t);
  if (getsz(t->c[0]) >= sz) {
    auto p = splitsz(t->c[0],sz); t->c[0] = p.second;
    return {p.first,calc(t)};
  } else {
    auto p=splitsz(t->c[1],sz-getsz(t->c[0])-1); t->c[1]=p.first;
    return {calc(t),p.second};
  }
}
pt merge(pt l, pt r) { //  keys in l < keys in r
  if (!l || !r) return l?:r;
  prop(l), prop(r); pt t;
  if (l->pri > r->pri) l->c[1] = merge(l->c[1],r), t = l;
  else r->c[0] = merge(l,r->c[0]), t = r;
  return calc(t);
}
pt ins(pt x, int v) { // insert v
  auto a = split(x,v), b = split(a.second,v+1);
  return merge(a.first,merge(new Node(v),b.second)); }
pt del(pt x, int v) { // delete v
  auto a = split(x,v), b = split(a.second,v+1);
  return merge(a.first,b.second); }
pt inspos(pt t, pt n, int pos) {// insert so node is in pos (0-indexed)
  auto pa = splitsz(t, pos);
  return merge(merge(pa.first, n), pa.second); }
pt delpos(pt t, int pos) {
  auto pa = splitsz(t, pos);
  auto pb = splitsz(pa.second, 1);
  return merge(pa.first, pb.second); }
```

UnionFind.h

Description: Disjoint-set data structure.
Time: $\mathcal{O}(\alpha(N))$ 7aa27c, 14 lines

```
struct UF {
  vi e;
  UF(int n) : e(n, -1) {}
  bool sameSet(int a, int b) { return find(a) == find(b); }
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : e[x] = find(e[x]); }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$ de4ad0, 21 lines

```
struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

Numerical (4)

4.1 Polynomials and recurrences

PolyRoots.h

Description: Finds the real roots to a polynomial.
Usage: `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`
Time: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$
"Polynomial.h" b00bfe, 23 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      }
      ret.push_back((l + h) / 2);
    }
  }
  return ret;
}
```

PolyOps.h

Description: Operations on formal power series c612fc, 150 lines

```
int const RT = 5;
using T = mint;
using poly = vector<mint>;
void remz(poly& p) { while (sz(p)&&p.back().v==0) p.pop_back(); }
poly REMZ(poly p) { remz(p); return p; }
poly rev(poly p) { reverse(all(p)); return p; }
poly shift(poly p, int x) {
  if (x >= 0) p.insert(begin(p),x,0);
  else assert(sz(p)+x >= 0), p.erase(begin(p),begin(p)-x);
  return p;
}
poly RSZ(const poly& p, int x) {
  if (x <= sz(p)) return poly(begin(p),begin(p)+x);
  poly q = p; q.resize(x); return q;  }
T eval(const poly& p, T x) { // evaluate at point x
  T res = 0; for (int i = sz(p)-1; i>=0; i--) res = x*res+p[i];
  return res; }
poly dif(const poly& p) { // differentiate
  poly res; rep(i,1,sz(p)) res.pb(T(i)*p[i]);
  return res; }
poly integ(const poly& p) { // integrate
  static poly invs{0,1};
  for (int i = sz(invs); i <= sz(p); ++i)
    invs.pb(-MOD/i*invs[MOD%i]);
  poly res(sz(p)+1); rep(i,0,sz(p)) res[i+1] = p[i]*invs[i+1];
  return res;
}
poly& operator+=(poly& l, const poly& r) {
  l.resize(max(sz(l),sz(r))); rep(i,0,sz(r)) l[i] += r[i];
  return l; }
poly& operator-=(poly& l, const poly& r) {
  l.resize(max(sz(l),sz(r))); rep(i,0,sz(r)) l[i] -= r[i];
  return l; }
poly& operator*=(poly& l, const T& r) { for (auto &t:l) t *= r;
  return l;  }
poly& operator/=(poly& l, const T& r) { for (auto &t:l) t /= r;
  return l; }
poly operator+(poly l, const poly& r) { return l += r; }
poly operator-(poly l, const poly& r) { return l -= r; }
poly operator-(poly l) { for (auto &t:l) t *= -1; return l; }
poly operator*(poly l, const T& r) { return l *= r; }
poly operator*(const T& r, const poly& l) { return l*r; }
poly operator/(poly l, const T& r) { return l /= r;  } // 8d2d0d
void fft(vector<T>& A, bool inverse = 0) { // NTT
  int n = sz(A); assert((MOD-1)%n == 0); vector<T> B(n);
  for(int b = n/2; b; b /= 2, swap(A,B)) { // w = n/b'th root
    T w = pow(mint(RT),(MOD-1)/n*b), m = 1;
    for(int i = 0; i < n; i += b*2, m *= w) rep(j,0,b) {
      T u = A[i+j], v = A[i+j+b]*m;
      B[i/2+j] = u+v; B[i/2+j+n/2] = u-v;
    }
  }
  if (inverse) { reverse(1+all(A));
    T z = invert(T(n)); for (auto &t:A) t *= z; }
} // for NTT-able moduli -- 3397f7
```

```
vector<T> conv(vector<T> A, vector<T> B) {
  if (!min(sz(A),sz(B))) return {};
  int s = sz(A)+sz(B)-1, n = 1; for (; n < s; n *= 2);
  A.resize(n), fft(A); B.resize(n), fft(B);
  rep(i,0,n) A[i] *= B[i];
  fft(A,1); A.resize(s); return A;
} // 7956e1
poly operator*(const poly& l, const poly& r) {
  assert(MOD == 998244353); return conv(l,r);
} // b6ba5b
// poly operator*(const poly& l, const poly& r) { // O(NM)
//   if (!min(sz(l),sz(r))) return {};
//   poly x(sz(l)+sz(r)-1);
//   rep(i,0,sz(l)) rep(j,0,sz(r)) x[i+j] += l[i]*r[j];
//   return x;
// }
poly& operator*=(poly& l, const poly& r) { return l = l*r; }
poly inv(poly A, int n) { // Q-(1/Q-A)/(-Q^{-2})
  poly B{invert(A[0])};
  for (int x = 2; x/2 < n; x *= 2)
    B = 2*B-RSZ(conv(RSZ(A,x),conv(B,B)),x);
  return RSZ(B,n);
} // 455ce0
poly sqrt(const poly& A, int n) {  // Q-(Q^2-A)/(2Q)
  assert(A[0].v == 1); poly B{1};
  for (int x = 2; x/2 < n; x *= 2)
    B = invert(T(2))*RSZ(B+conv(RSZ(A,x),inv(B,x)),x);
  return RSZ(B,n);
} // ecb486
// return {quotient, remainder}
pair<poly,poly> quoRem(const poly& f, const poly& g) {
  if (sz(f) < sz(g)) return {{},f};
  poly q = conv(inv(rev(g),sz(f)-sz(g)+1),rev(f));
  q = rev(RSZ(q,sz(f)-sz(g)+1));
  poly r = RSZ(f-conv(q,g),sz(g)-1); return {q,r};
} // bffc5b
poly log(poly A, int n) { assert(A[0].v == 1); // (ln A)' = A'/A
  A.resize(n); return integ(RSZ(conv(dif(A),inv(A,n-1)),n-1),n-1);
} // bda418
poly exp(poly A, int n) { assert(A[0].v == 0);
  poly B{1}, IB{1}; // inverse of B
  for (int x = 1; x < n; x *= 2) {
    IB = 2*IB-RSZ(conv(B,conv(IB,IB)),x);
    poly Q = dif(RSZ(A,x)); Q += RSZ(conv(IB,dif(B)-conv(B,Q)),2*x-1);
    B = B+RSZ(conv(B,RSZ(A,2*x)-integ(Q)),2*x);
  }
  return RSZ(B,n);
} // 203953
poly pow(poly A, ll b, int n) { // A^b mod x^n
  A.resize(n); int t = 0; while (t < n && !A[t].v) t++;
  if (!b) { poly r(n); r[0] = 1; return r; }
  if (t == n || (t && b >= n) || t*b >= n) return poly(n);
  int k = int(n-t*b); mint c = A[t];
  poly p = RSZ(poly(begin(A)+t, end(A))/c, k);
  p = exp(log(p,k)*mint(b),k)*pow(c,b);
  p.insert(begin(p), t*b, 0); return p;
} // d0547e
poly mod(const poly& f, const poly& g) { return quoRem(f,g).second; }
poly xkmodf(ll k, poly f) {
    poly r{1}, a{0,1};
    for(;k;k>>=1) {
        if(k&1) r = mod(conv(r,a), f);
        a = mod(conv(a,a), f);
    }
    return r;
} // ef2278
// [x^k] P/Q in O(d log d log k), Q[0] != 0, deg P < deg Q = d
T kthCoef(poly P, poly Q, ll k) {
  for (; k; k /= 2) {
    poly R = Q; for (int i = 1; i < sz(R); i += 2) R[i] = -R[i];
    poly A = conv(P,R), B = conv(Q,R); P.clear(), Q.clear();
    for (int i = k&1; i < sz(A); i += 2) P.pb(A[i]);
    for (int i = 0; i < sz(B); i += 2) Q.pb(B[i]);
  }
  return sz(P) ? P[0]/Q[0] : 0;
} // 993413
// solve recurrence with initial vals s[0], s[1]... s[n-1]
// a[k] = c[1]*a[k-1] + c[2]*a[k-2] + ... c[n]*a[k-n]
mint solve_linrec(vector<mint> s, vector<mint> c, int n, ll k) {
    poly f(n+1, 0);
    f[n] = 1;
    for (int i=0;i<n;i++) f[i] = mint(-c[n-i]);
    poly r = xkmodf(k, f); r.resize(n);
    mint ans(0);
    for (int i = 0; i < n; i++) ans += r[i] * mint(s[i]);
    return ans;
} // 902e38
// returns f(x+c)
poly taylorShift(poly f, T c) {
  int n = sz(f); poly F(n+1,1), A(n), B(n); T p = 1;
  rep(i,1,n+1) F[i] = F[i-1]*T(i);
  rep(i,0,n) A[n-1-i] = f[i]*F[i], B[i] = p/F[i], p *= c;
  A = conv(A,B); rep(i,0,n) f[i] = A[n-1-i]/F[i];
  return f;
} // 293ba2
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.

Time: $\mathcal{O}\left(n^2\right)$

dc6c70, 14 lines

```
typedef vector<double> vd;
template<class T>
vector<T> interpolate(vector<T> x, vector<T> y, int n) {
  vector<T> res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  T last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```

PolyMultipoint.h

Description: Multipoint evaluation and interpolation

Time: $\mathcal{O}\left(N \log^2 N\right)$

238885, 30 lines

```
void segProd(vector<poly>& stor, poly& v, int ind, int l,
    int r) { // v -> places to evaluate at
  if (l == r) { stor[ind] = {-v[l],1}; return; }
  int m = (l+r)/2; segProd(stor,v,2*ind,l,m); segProd(stor,v
    ,2*ind+1,m+1,r);
  stor[ind] = conv(stor[2*ind],stor[2*ind+1]);
}
void evalAll(vector<poly>& stor, poly& res, poly v, int ind
    = 1) {
  v = quoRem(v,stor[ind]).second;
  if (sz(stor[ind]) == 2) { res.pb(sz(v)?v[0]:0); return; }
  evalAll(stor,res,v,2*ind); evalAll(stor,res,v,2*ind+1);
}
// evaluate polynomial v at points in p
poly multiEval(poly v, poly p) {
  if (p.empty()) return {};
  vector<poly> stor(4*sz(p)); segProd(stor,p,1,0,sz(p)-1);
  poly res; evalAll(stor,res,v); return res; }

poly combAll(vector<poly>& stor, poly& dems, int ind, int l,
    int r) {
  if (l == r) return {dems[l]};
  int m = (l+r)/2;
  poly a = combAll(stor,dems,2*ind,l,m), b = combAll(stor,
    dems,2*ind+1,m+1,r);
  return conv(a,stor[2*ind+1])+conv(b,stor[2*ind]);
}
poly interpolate(vector<pair<T,T>> v) {
  int n = sz(v); poly x; for (auto t:v) x.pb(t.first);
  if (!n) return {};
  vector<poly> stor(4*n); segProd(stor,x,1,0,n-1);
  poly dems; evalAll(stor,dems,dif(stor[1]));
  rep(i,0,n) dems[i] = v[i].second/dems[i];
  return combAll(stor,dems,1,0,n-1);
}
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.

Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`

Time: $\mathcal{O}\left(N^2\right)$

"../number-theory/ModPow.h" 84669d, 20 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n+1), B(n+1), T;
  C[0] = B[0] = 1;

  ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }

  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
}
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.

Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`

Time: $\mathcal{O}\left(n^2 \log k\right)$

3b95b4, 26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr); if (!n) return 0;

  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) %
       mod;
    res.resize(n + 1);
    return res;
  };

  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }

  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}
```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum. The maximum error in the result is eps. Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.

Usage: `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func);`

Time: $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$

31d45b, 14 lines

```
double gss(double a, double b, double (*f)(double)) {
  double r = (sqrt(5)-1)/2, eps = 1e-7;
  double x1 = b - r*(b-a), x2 = a + r*(b-a);
  double f1 = f(x1), f2 = f(x2);
  while (b-a > eps)
```

```
    if (f1 < f2) { //change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r*(b-a); f1 = f(x1);
    } else {
      a = x1; x1 = x2; f1 = f2;
      x2 = a + r*(b-a); f2 = f(x2);
    }
  return a;
}
```

HillClimbing.h

Description: Poor man's optimization for unimodal functions. 8eeeaf, 14 lines

```
typedef array<double, 2> P;

template<class F> pair<double, P> hillClimb(P start, F f) {
  pair<double, P> cur(f(start), start);
  for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
    rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
      P p = cur.second;
      p[0] += dx*jmp;
      p[1] += dy*jmp;
      cur = min(cur, make_pair(f(p), p));
    }
  }
  return cur;
}
```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4, although in practice you will want to verify that the result is stable to desired precision when epsilon changes. 4756fc, 7 lines

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
  double h = (b - a) / 2 / n, v = f(a) + f(b);
  rep(i,1,n*2)
    v += f(a + i*h) * (i&1 ? 4 : 2);
  return v * h / 3;
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.
Usage: `double sphereVolume = quad(-1, 1, [](double x) {`
`return quad(-1, 1, [&](double y) {`
`return quad(-1, 1, [&](double z) {`
`return x*x + y*y + z*z < 1; });});});` 9f06d6, 15 lines

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || abs(b - a) < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S
    2);
}
template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage: `vvd A = {{1,-1}, {-1,1}, {-1,-2}};`
`vd b = {1,1,-4}, c = {-1,-1}, x;`
`T val = LPSolver(A, b, c).solve(x);`
Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case. aa8530, 68 lines

```
typedef double T; // long double, Rational, double + mod<P
    >...
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s]))
    s=j

struct LPSolver {
  int m, n;
  vi N, B;
  vvd D;

  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i
    ];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
    }

  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      rep(j,0,n+2) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }

  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }

  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
        rep(j,1,n+1) ltj(D[i]);
        pivot(i, s);
      }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  }
};
```

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}\left(N^3\right)$ 4583fb, 16 lines

```
template<class T>
T det(vector<vector<T>>& a) {
  int n = sz(a); T res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (abs(a[j][i]) > abs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      T v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}\left(N^3\right)$ 3313dc, 18 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}\left(n^2 m\right)$ 580b91, 41 lines

```
typedef vector<double> vd;
const double eps = 1e-12;

template<class T>
int solveLinear(vector<vector<T>>& A, vector<T>& b, vector<T
    >& x) {
  T tol = 0;
  if constexpr (is_floating_point<T>::value) tol = T(eps);
  int n = sz(A), m = sz(x), rank = 0, br = 0, bc = 0;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);

  rep(i,0,n) {
    T v, bv = 0;
    rep(r,i,n) rep(c,i,m)
      if ((v = abs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= tol) {
      rep(j,i,n) if (abs(b[j]) > tol) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      T fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    }
    rank++;
  }

  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

"SolveLinear.h" 08e495, 7 lines

```
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
  rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b.

Time: $\mathcal{O}\left(n^2m\right)$ fa2d7a, 34 lines

```
typedef bitset<1000> bs;

int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x));
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
      rep(j,i,n) if(b[j]) return -1;
      break;
    }
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    }
    rep(j,i+1,n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  }

  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

XorBasis.h

Description: Basis of a set of 64-bit vectors over $\mathbb{F}_2$. $b[i]$ is 0 or a basis vector with highest set bit i; r is the rank, so 2^r distinct values are representable as xors of subsets (0 always is). add returns true iff x was not representable before. maxXor/minXor give the max/min of $x \oplus v$ over representable v. kth returns the k-th smallest representable value, 0-indexed (`kth(0)` $= 0$), and requires $k < 2^r$. Values are compared as unsigned; cast negative numbers to ull.

Usage:
```
XorBasis B; B.add(5); B.add(3);
B.has(6); // true
B.maxXor(); B.kth(2); // 6, 5
```

Time: $\mathcal{O}(64)$ per operation, $\mathcal{O}\left(64^2\right)$ for merge. e3100d, 27 lines

```
typedef unsigned long long ull;
struct XorBasis {
  ull b[64] = {}; int r = 0;
  bool add(ull x) {
    for (int i = 64; i--;) if (x >> i & 1) {
      if (!b[i]) return b[i] = x, ++r;
      x ^= b[i];
    }
    return 0;
  }
  ull minXor(ull x) {
    for (int i = 64; i--;) x = min(x, x ^ b[i]);
    return x;
  }
  ull maxXor(ull x = 0) {
    for (int i = 64; i--;) x = max(x, x ^ b[i]);
    return x;
  }
  bool has(ull x) { return !minXor(x); }
  ull kth(ull k) {
    ull x = 0; int c = r;
    for (int i = 64; i--;) if (b[i])
      if ((x >> i ^ k >> --c) & 1) x ^= b[i];
    return x;
  }
  void merge(const XorBasis& o) { for (ull x : o.b) add(x); }
};
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.

Time: $\mathcal{O}\left(n^3\right)$ fe5d0b, 38 lines

```
using T = double; // or mint
template<class T>
int matInv(vector<vector<T>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<T>> tmp(n, vector<T>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (abs(A[j][k]) > abs(A[r][c]))
        r = j, c = k;
    bool zero = A[r][c] == 0;
    if constexpr (is_floating_point<T>::value)
      zero = abs(A[r][c]) < 1e-12;
    if (zero) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    T v = A[i][i];
    rep(j,i+1,n) {
      T f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  }
  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    T v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  }
  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

Gaussian.h

Description: Converts A to rref using Gaussian elim.

Time: $\mathcal{O}\left(N^3\right)$ 462ea6, 32 lines

```
using T = double; // or mint
const double EPS = 1e-9; // adjust?
template<class T>
int getRow(vector<vector<T>>& m, int R, int i, int nex) {
  if constexpr (is_floating_point<T>::value) {
    pair<T,int> best{0,-1};
    rep(j,nex,R) best = max(best, {abs(m[j][i]), j});
    return best.first < EPS ? -1 : best.second;
  } else {
    rep(j,nex,R) if (m[j][i] != 0) return j;
    return -1;
  }
}
template<class T>
pair<T,int> gauss(vector<vector<T>>& m) { // convert to rref
  if (!sz(m)) return {1,0};
  int R = sz(m), C = sz(m[0]), rank = 0, nex = 0;
  T prod = 1; // determinant
  rep(i,0,C) {
    int row = getRow(m,R,i,nex);
    if (row == -1) { prod = 0; continue; }
    if (row != nex) prod *= -1, swap(m[row],m[nex]);
    prod *= m[nex][i]; rank++;
    T x = 1/m[nex][i]; rep(k,i,C) m[nex][k] *= x;
    rep(j,0,R) if (j != nex) {
      T v = m[j][i]; if (v == 0) continue;
      rep(k,i,C) m[j][k] -= v*m[nex][k];
    }
    nex++;
  }
  return {prod,rank};
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \ldots, -1, 1\}, \{0, c_1, c_2, \ldots, c_n\}, \{b_1, b_2, \ldots, b_n, 0\}, \{a_0, -d_1, -d_2, \ldots, -d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i, or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.

Time: $\mathcal{O}(N)$ 873844, 30 lines

```
typedef double T;
template<class T>
vector<T> tridiagonal(vector<T> diag, const vector<T>& super
    ,
    const vector<T>& sub, vector<T> b) {
  int n = sz(b); vi tr(n);
  rep(i,0,n-1) {
    bool zero = diag[i] == 0;
    if constexpr (is_floating_point<T>::value)
      zero = abs(diag[i]) < 1e-9 * abs(super[i]);
    if (zero) {
      b[i+1] -= b[i] * diag[i+1] / super[i];
      if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
      diag[i+1] = sub[i]; tr[++i] = 1;
    } else {
      diag[i+1] -= super[i]*sub[i]/diag[i];
      b[i+1] -= b[i]*sub[i]/diag[i];
    }
  }
  for (int i = n; i--;) {
    if (tr[i]) {
      swap(b[i], b[i-1]);
      diag[i-1] = diag[i];
      b[i] /= super[i-1];
    } else {
      b[i] /= diag[i];
      if (i) b[i-1] -= b[i]*super[i-1];
    }
  }
  return b;
}
```

4.4 Fourier transforms

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim$1s for $N = 2^{22}$) 00ced6, 35 lines

```
typedef complex<double> C;
```

```
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-
     rolled)
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
}
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

FFTComplex.h

Description: FFT but with complex numbers
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ (~1s for $N = 2^{22}$) 8cb345, 14 lines

```
vector<C> conv_complex(const vector<C>& a, const vector<C>&
    b) {
  if (a.empty() || b.empty()) return {};
  vector<C> res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in1(n), in2(n), out(n);
  copy(all(a), begin(in1));
  copy(all(b), begin(in2));
  fft(in1);
  fft(in2);
  rep(i,0,n) out[i] = conj(in1[i] * in2[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = conj(out[i]) / C(n, 0);
  return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)
"FastFourierTransform.h" b82773, 22 lines

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  rep(i,0,n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x] g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a. For arbitrary modulo, see FFTMod. conv(a, b) = c, where $c[x] = \sum a[i] b[x - i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod).
Time: $\mathcal{O}(N \log N)$
"../number-theory/ModPow.h" d54056, 33 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 <<
    21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};
    rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    }
}
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s), n =
     1 << B;
  ll inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i,0,n) out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv
      % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.
Time: $\mathcal{O}(N \log N)$ 464cf3, 16 lines

```
void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
        pii(u + v, u - v);                   // XOR
    }
  }
  if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
}
```

Number theory (5)

5.1 Modular arithmetic

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a.
Time: $\mathcal{O}(\sqrt{m})$ c040b8, 11 lines

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
modsum(to, c, k, m) $= \sum_{i=0}^{to-1} (ki + c) \% m$. divsum is similar but for floored division.
Time: $\log(m)$, with a large constant. 5c5bc5, 16 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModInt.h

Description: Modular integers for $1 \le M \le 2^{30}$. Keep v in [0,M). Division requires a divisor coprime to M; pow requires a nonnegative integer exponent. Stream input uses ll.
Usage: using mint = ModInt<998244353>;
Time: $\mathcal{O}(1)$, except $\mathcal{O}(\log M)$ division and $\mathcal{O}(\log b)$ power. a6d338, 72 lines

```
int const MOD = 998244353;
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}
template<int M> struct ModInt {
  static_assert(0 < M && M <= (1<<30));
  int v;
  constexpr explicit operator int() const {return v;}
  static constexpr int mod() {return M;}
  constexpr int val() const {return v;}
  constexpr ModInt(): v(0) {}
  constexpr ModInt(auto z) requires requires { z % M; }
    : v(int(z % M)) {
    if (v < 0) v += M;
  }
  static constexpr ModInt raw(int x) {
    assert(0 <= x && x < M);
    ModInt a; a.v = x; return a;
  }
  friend ModInt invert(ModInt a) {
    ll x, y, g = euclid(a.v, M, x, y);
    assert(g == 1); return ModInt(x);
  }
  constexpr ModInt& operator+= (ModInt const& o) {
    if ((v += o.v) >= M) v -= M;
    return *this;}
  constexpr ModInt& operator-= (ModInt const& o) {
    if ((v -= o.v) < 0) v += M;
    return *this;}
```

```
  constexpr ModInt& operator*= (ModInt const& o) {
    v = int((ll)v*o.v%M); return *this;}
  ModInt& operator/= (ModInt const& o) {
    return *this *= invert(o);}

  friend constexpr ModInt operator+ (ModInt a, ModInt const&
      b) {
    return a+=b;}
  friend constexpr ModInt operator- (ModInt a, ModInt const&
      b) {
    return a-=b;}
  friend constexpr ModInt operator* (ModInt a, ModInt const&
      b) {
    return a*=b;}
  friend ModInt operator/ (ModInt const& a, ModInt const& b)
      {
    return a*invert(b);}
  friend constexpr bool operator== (ModInt a, ModInt b) {
    return a.v == b.v;}
  friend constexpr bool operator!= (ModInt a, ModInt b) {
    return !(a == b);}

  constexpr ModInt operator+ () const {return *this;}
  constexpr ModInt operator- () const {return ModInt(-v);}
  constexpr ModInt& operator++ () {return *this += 1;}
  constexpr ModInt& operator-- () {return *this -= 1;}
  constexpr ModInt operator++ (int) {auto a = *this; ++*this
      ; return a;}
  constexpr ModInt operator-- (int) {auto a = *this; --*this
      ; return a;}
  friend constexpr ModInt pow(ModInt a, auto b) {
    assert(b >= 0);
    ModInt r(1);
    for(;b;b>>=1, a*=a)
      if(b&1)
        r *= a;
    return r;
  }
  friend istream& operator>> (istream& in, ModInt& a) {
    ll x; if (in >> x) a = ModInt(x);
    return in;
  }
  friend ostream& operator<< (ostream& out, ModInt a) {
    return out << a.v;
  }
};
using mint = ModInt<MOD>;
```

ModHelpers.h

Description: Computes inv, fact, ifact. Requires prime MOD and MV <= MOD. Call init before use; choose requires 0 <= k <= n < MV.
Time: $\mathcal{O}(N)$

"ModInt.h" e1c06f, 17 lines

```
int const MV = 2e6 + 10;
static_assert(MV <= MOD);
mint inv[MV], fact[MV], ifact[MV];
void init() {
  inv[1] = mint(1);
  for(int i = 2; i < MV; ++i)
    inv[i] = mint(MOD - MOD/i) * inv[MOD % i];
  fact[0] = ifact[0] = mint(1);
  for(int i = 1; i < MV; ++i) {
    fact[i] = mint(i) * fact[i-1];
    ifact[i] = inv[i] * ifact[i-1];
  }
}
mint choose(int n, int k) {
  assert(0 <= k && k <= n && n < MV); // or return 0
  return fact[n] * ifact[n-k] * ifact[k];
}
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \le a, b \le c \le 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for `modmul`, $\mathcal{O}(\log b)$ for `modpow`

bbbd8f, 11 lines

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution). Requires p prime, $p < 3 \cdot 10^9$ (use modmul for the products if p is larger).
Time: $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModMulLL.h" 500b57, 24 lines

```
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2, p) == 1); // else no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (modpow(n, (p - 1) / 2, p) != ull(p - 1)) ++n;
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 ≈ 1.5s

6b2912, 20 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1))
    ;
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    rep(i,0,min(S, R - L))
      if (!block[i]) pr.push_back((L + i) * 2 + 1);
  }
  for (int i : pr) isPrime[i] = 1;
  return pr;
}
```

PrimeSieve.h

Description: Prime sieve but slow, for generating all primes smaller than LIM.
Time: LIM=1e7 ≈ 0.15s, LIM=1e8 ≈ 2s. Uses ≈ 12 bytes per number (1.2GB for LIM=1e8).

917bd8, 17 lines

```
int const LIM = 1e7+5;
vector<bool> cp;
vi pr, nx, lp, cnt;
void sieve()
{
  pr.clear(), cp.assign(LIM, 0), nx.assign(LIM, -1), lp.
     assign(LIM, -1), cnt.assign(LIM, -1);
  for(int i=2;i<LIM;++i) {
    if(!cp[i])
      lp[i] = pr.size(), nx[i] = cnt[i] = 1, pr.push_back(i)
      ;
    for(int j=0,k;j<pr.size() && (k=i*pr[j])<LIM;++j) { //
     pr[j]<(LIM+i-1)/i, if there's overflow
      cp[k] = 1, lp[k] = j;
      if(j == lp[i]) {
        nx[k] = nx[i], cnt[k] = cnt[i]+1; break;
      } else nx[k] = i, cnt[k] = 1;;
    }
  }
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \mod c$.

"ModMulLL.h" 60dcd1, 12 lines

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 179526502
     2},
       s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d8d98d, 18 lines

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

"euclid.h" 04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

5.3.1 Bézout's identity

For $a \neq, b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler's* ϕ function is defined as $\phi(n) :=$ # of positive integers $\leq n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$ m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} ... (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$. $\sum_{d|n} \phi(d) = n$, $\sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$

Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod{n}$.

Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod{p}$ $\forall a$.

cf7d6d, 8 lines

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

Fraction.h

Description: Exact finite fractions, stored as coprime p/q with q > 0. Uses long long storage and signed 128-bit intermediates. Throws overflow_error if the reduced numerator or positive denominator does not fit, and domain_error on division by zero. Do not modify p or q directly. Floating conversions are explicit; no epsilon is needed. Input accepts integer or p/q tokens; output is always p/q. fromWide(p,q) reduces a signed 128-bit ratio before checking storage.

Usage: `using P = Point<Fraction>;`
`Fraction x(1, 3); // x + 2 == Fraction(7, 3)`

Time: $\mathcal{O}(logM)$ per arithmetic operation (128-bit Euclid).

1a036c, 75 lines

```
struct Fraction {
  using I = __int128_t;
  ll p = 0, q = 1;
  Fraction(ll a = 0, ll b = 1) { *this = make(a, b); }
  static Fraction fromWide(I p, I q = 1) { return make(p, q)
    ; }
  template<class A, class B = int, enable_if_t<
    is_floating_point<A>::value ||
    is_floating_point<B>::value, int> = 0>
  Fraction(A, B = 1) = delete;

  explicit operator bool() const { return p != 0; }
  explicit operator double() const {
    return (double)p / (double)q; }
  explicit operator long double() const {
    return (long double)p / q; }
  Fraction operator+() const { return *this; }
  Fraction operator-() const { return make(-I(p), q); }
  friend Fraction operator+(Fraction a, Fraction b) {
    return make(I(a.p)*b.q + I(b.p)*a.q, I(a.q)*b.q); }
  friend Fraction operator-(Fraction a, Fraction b) {
    return make(I(a.p)*b.q - I(b.p)*a.q, I(a.q)*b.q); }
  friend Fraction operator*(Fraction a, Fraction b) {
    return make(I(a.p)*b.p, I(a.q)*b.q); }
  friend Fraction operator/(Fraction a, Fraction b) {
    return make(I(a.p)*b.q, I(a.q)*b.p); }
  Fraction& operator+=(Fraction b) { return *this = *this+b;
    }
  Fraction& operator-=(Fraction b) { return *this = *this-b;
    }
  Fraction& operator*=(Fraction b) { return *this = *this*b;
    }
  Fraction& operator/=(Fraction b) { return *this = *this/b;
    }
  friend bool operator==(Fraction a, Fraction b) {
    return a.p == b.p && a.q == b.q; }
  friend bool operator!=(Fraction a, Fraction b) {
    return !(a == b); }
  friend bool operator<(Fraction a, Fraction b) {
    return I(a.p)*b.q < I(b.p)*a.q; }
  friend bool operator>(Fraction a, Fraction b) { return b<a
    ; }
  friend bool operator<=(Fraction a, Fraction b) {
    return !(b < a); }
  friend bool operator>=(Fraction a, Fraction b) {
    return !(a < b); }
  friend Fraction abs(Fraction a) { return a.p < 0 ? -a : a;
    }
  friend ostream& operator<<(ostream& os, Fraction a) {
    return os << a.p << '/' << a.q; }
  friend istream& operator>>(istream& is, Fraction& a) {
    string s;
    if (!(is >> s)) return is;
    istringstream in(s);
    ll p, q = 1; char c;
    if (!(in >> p) || (in >> c &&
      (c != '/' || !(in >> q))) || !in.eof())
      is.setstate(ios::failbit);
    else {
      try { a = Fraction(p, q); }
      catch (const domain_error&) { is.setstate(ios::failbit
      ); }
      catch (const overflow_error&) { is.setstate(ios::
      failbit); }
    }
    return is;
  }
private:
  struct Raw {};
  Fraction(ll p, ll q, Raw) : p(p), q(q) {}
  static Fraction make(I p, I q) {
    if (!q) throw domain_error("Fraction: zero denominator")
    ;
    using U = __uint128_t;
    bool neg = (p < 0) != (q < 0);
    U n = p < 0 ? -U(p) : U(p), d = q < 0 ? -U(q) : U(q);
    U a = n, b = d;
    while (b) { U r = a % b; a = b; b = r; }
    n /= a; d /= a;
    if (n > U(LLONG_MAX) + neg || d > U(LLONG_MAX))
      throw overflow_error("Fraction: reduced value out of
      range");
    ll num = neg && n ? -ll(n-1)-1 : ll(n);
    return Fraction(num, (ll)d, Raw{});
  }
};
```

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.

For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. $(p_k/q_k$ alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞; if x is the root of a degree 2 polynomial the a's eventually become cyclic.

Time: $\mathcal{O}(\log N)$

dd6c5e, 21 lines

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
  for (;;) {
    ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf)
    ,
      a = (ll)floor(y), b = min(a, lim),
      NP = b*P + LP, NQ = b*Q + LQ;
    if (a > b) {
      // If b > a/2, we have a semi-convergent that gives us
      a
      // better approximation; if b = a/2, we *may* have one
      .
      // Return {P, Q} here for a more canonical
      approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q))
      ?
        make_pair(NP, NQ) : make_pair(P, Q);
    }
    if (abs(y = 1/(y - (d)a)) > 3*N) {
      return {NP, NQ};
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
  }
}
```

FracBinarySearch.h

Description: Given f and N, finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N \leq 3 \cdot 10^{18}$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.

Usage: `fracBS([](Frac f) { return 3*f.p>=f.q; }, 10); // {1,3}`

Time: $\mathcal{O}(\log(N))$

27ab3e, 25 lines

```
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
  if (f(lo)) return lo;
  assert(f(hi));
  while (A || B) {
    ll adv = 0, step = 1; // move hi if dir, else lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step; si = 2;
      }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \ \ b = k \cdot (2mn), \ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

5.9 Lifting the Exponent

For $n > 0$, p prime, and ints x, y s.t. $p \nmid x, y$ and $p | x - y$:

- $p \neq 2$ or $p = 2, 4 | x - y \implies v_p(x^n - y^n) = v_p(x - y) + v_p(n)$.
- $p = 2, 2 | n \implies v_2(x^n - y^n) = v_2((x^2)^{n/2} - (y^2)^{n/2})$.

Combinatorial (6)

6.1 Permutations

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$ 044568, 6 lines

```
int permToInt(vi& v) {
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
    use |= 1 << x;                    // (note: minus, not ~!)
  return r;
}
```

6.1.1 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.2 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

6.1.3 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

6.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod{p}$.

6.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1 + \cdots + k_n}{k_1, k_2, \ldots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \ldots k_n!}$. b00768, 6 lines

```
ll multinomial(vi& v) {
  ll c = 1, m = v.empty() ? 1 : v[0];
  rep(i,1,sz(v)) rep(j,0,v[i])
    c = ll((__int128)c * ++m / (j+1));
  return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).

$B[0, \ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1} \sum_{k=0}^{m} \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$
$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\sum_{k=0}^{n} c(n,k) x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \ge j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$
$$E(n,0) = E(n,n-1) = 1$$
$$E(n,k) = \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$
$$S(n,1) = S(n,n) = 1$$
$$S(n,k) = \frac{1}{k!} \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i: $(n-2)!/((d_1-1)! \cdots (d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2} C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

6.4 Young Tableaux

Let a **Young diagram** have shape $\lambda = (\lambda_1 \ge \cdots \ge \lambda_k)$, where λ_i equals the number of cells in the i-th (left-justified) row from the top. A **Young tableau** of shape λ is a filling of the $n = \sum \lambda_i$ cells with a permutation of $1 \ldots n$ such that each row and column is increasing.

Hook-Length Formula: For the cell in position (i,j), let $h_\lambda(i,j) = |\{(I,J) | i \le I, j \le J, (I = i \text{ or } J = j)\}|$. The number of Young tableaux of shape λ is equal to $f^\lambda = \frac{n!}{\prod h_\lambda(i,j)}$.

Schensted's Algorithm: converts a permutation σ of length n into a pair of Young Tableaux $(S(\sigma), T(\sigma))$ of the same shape. When inserting $x = \sigma_i$,

1. Add x to the first row of S by inserting x in place of the largest y with $x < y$. If y doesn't exist, push x to the end of the row, set the value of T at that position to be i, and stop.
2. Add y to the second row using the same rule, keep repeating as necessary.

For $\sigma = (5, 2, 3, 1, 4)$,

$$S(\sigma): \quad 5 \to \begin{matrix}2\\5\end{matrix} \to \begin{matrix}2 & 3\\5\end{matrix} \to \begin{matrix}1 & 3\\2\\5\end{matrix} \to \begin{matrix}1 & 3 & 4\\2\\5\end{matrix}$$

$$T(\sigma): \quad 1 \to \begin{matrix}1\\2\end{matrix} \to \begin{matrix}1 & 3\\2\end{matrix} \to \begin{matrix}1 & 3\\2\\4\end{matrix} \to \begin{matrix}1 & 3 & 5\\2\\4\end{matrix}$$

All pairs $(S(\sigma), T(\sigma))$ of the same shape correspond to a unique σ, so $n! = \sum (f^\lambda)^2$. Also, $S(\sigma^R) = S(\sigma)^T$.

Let $d_k(\sigma), a_k(\sigma)$ be the lengths of the longest subseqs which are a union of k decreasing/ascending subseqs, respectively. Then $a_k(\sigma) = \sum_{i=1}^{k} \lambda_i, d_k(\sigma) = \sum_{i=1}^{k} \lambda_i^*$, where λ_i^* is size of the i-th column.

6.4.1 Prufer Sequences

Tree to seq: Take lowest label leaf, print label of neighbour, until 2 nodes left

Seq to tree: deg of each node = frequency in sequence+1. take lowest 1-degree node connect it to first value in sequence, increase sequence pointer. repeat.

6.5 Other

DeBruijnSeq.h

Description: Given alphabet $[0, k)$ constructs a cyclic string of length k^n that contains every length n string as substr. 2dfa89, 13 lines

```
vi deBruijnSeq(int k, int n) {
  if (k == 1) return {0};
  vi seq, aux(n+1);
  function<void(int,int)> gen = [&](int t, int p) {
    if (t > n) { // +lyndon word of len p
      if (n%p == 0) rep(i,1,p+1) seq.pb(aux[i]);
    } else {
      aux[t] = aux[t-p]; gen(t+1,p);
      while (++aux[t] < k) gen(t+1,t);
    }
  };
  gen(1,1); return seq;
}
```

NimProduct.h

Description: Product of nimbers is associative, commutative, and distributive over addition (xor). Forms finite field of size 2^{2^k}. Defined by $ab = \text{mex}(\{a'b + ab' + a'b' : a' < a, b' < b\})$. Application: Given 1D coin turning games G_1, G_2 $G_1 \times G_2$ is the 2D coin turning game defined as follows. If turning coins at $x_1, x_2, \ldots, x_m$ is legal in G_1 and $y_1, y_2, \ldots, y_n$ is legal in G_2, then turning coins at all positions (x_i, y_j) is legal assuming that the coin at (x_m, y_n) goes from heads to tails. Then the grundy function $g(x, y)$ of $G_1 \times G_2$ is $g_1(x) \times g_2(y)$.

Time: 64^2 xors per multiplication, memorize to speed up. e286be, 45 lines

```
using ul = uint64_t;
struct Precalc {
  ul tmp[64][64], y[8][8][256];
  unsigned char x[256][256];
  Precalc() { // small nim products, all < 256
    rep(i,0,256) rep(j,0,256) x[i][j] = mult<8>(i,j);
    rep(i,0,8) rep(j,0,i+1) rep(k,0,256)
      y[i][j][k] = mult<64>(prod2(8*i,8*j),k);
  }
  ul prod2(int i, int j) { // nim prod of 2^i, 2^j
    ul& u = tmp[i][j]; if (u) return u;
    if (!(i&j)) return u = 1ULL<<(i|j);
    int a = (i&j)&-(i&j); // a=2^k, consider 2^{2^k}
    return u=prod2(i^a,j)^prod2((i^a)|(a-1),(j^a)|(i&(a-1)))
     ;
    // 2^{2^k}*2^{2^k} = 2^{2^k}+2^{2^k-1}
  } // 2^{2^i}*2^{2^j} = 2^{2^i+2^j} if i<j
  template<int L> ul mult(ul a, ul b) {
    ul c = 0; rep(i,0,L) if (a>>i&1)
      rep(j,0,L) if (b>>j&1) c ^= prod2(i,j);
    return c;
  }
  // 2^{8*i}*(a>>(8*i)&255) * 2^{8*j}*(b>>(8*j)&255)
  // -> (2^{8*i}*2^{8*j})*((a>>(8*i)&255)*(b>>(8*j)&255))
  ul multFast(ul a, ul b) const { // faster nim product
    ul res = 0; auto f=[](ul c,int d) {return c>>(8*d)&255
     ;};
    rep(i,0,8) {
      rep(j,0,i) res ^= y[i][j][x[f(a,i)][f(b,j)]
                    ^x[f(a,j)][f(b,i)]];
      res ^= y[i][i][x[f(a,i)][f(b,i)]];
    }
    return res;
  }
};
const Precalc P;
```

```
struct nb { // nimber
  ul x; nb() { x = 0; }
  nb(ul _x): x(_x) {}
  explicit operator ul() { return x; }
  nb operator+(nb y) { return nb(x^y.x); }
  nb operator*(nb y) { return nb(P.multFast(x,y.x)); }
  friend nb pow(nb b, ul p) {
    nb res = 1; for (;p;p/=2,b=b*b) if (p&1) res = res*b;
    return res; } // b^{2^{2^A}-1}=1 where 2^{2^A} > b
  friend nb inv(nb b) { return pow(b,-2); }
};
```

Graph (7)

7.1 Network flow

MinCostMaxFlow.h

Description: Min-cost max-flow. Multi-edges are allowed. An edge stores flow = -(residual capacity), so the flow sent over an added edge e (not a self-loop) is -ed[e.n][e.rev].flow. If costs can be negative, call setpi before maxflow; negative cost cycles are not supported. With C = F = int, every shortest path cost and every difference of two such costs (these are the reduced distances) must stay below INFC ($\approx 2^{29}$), and the total flow below 2^{31}; otherwise change the typedefs to ll.

Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi. 4f8c64, 66 lines

```
// #include <bits/extc++.h>
struct MCMF {
  typedef int C; typedef int F; typedef ll R;
  C const INFC = numeric_limits<C>::max() / 4;
  F const INFF = numeric_limits<F>::max() / 4;

  struct Edge {int n; F flow; C cost; size_t rev;};
  int N;
  vector<vector<Edge>> ed;
  vector<C> dist, pi;
  vector<F> amt;
  vector<size_t> par;

  MCMF(int N) :
    N(N), ed(N), dist(N), pi(N), amt(N), par(N) {}

  void addEdge(int u, int v, F f, C c) {
    ed[u].push_back({v, -f, c, ed[v].size()});
    ed[v].push_back({u, 0, -c, ed[u].size()-1});
  }

  void path(int s) { // 417ab0
    fill(all(amt), 0); amt[s] = INFF;
    fill(all(dist), INFC); dist[s] = 0;

    __gnu_pbds::priority_queue<pair<C, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({0, s});

    while (!q.empty()) {
      s = q.top().second; q.pop();
      C di = dist[s] + pi[s];
      for (auto [n, f, c, rev]: ed[s])
        if (f < 0 && ckmin(dist[n], di + c - pi[n])) {
          par[n] = rev; amt[n] = min(amt[s], -f);
          if(its[n] == q.end()) its[n] = q.push({-dist[n], n
     });
          else q.modify(its[n], {-dist[n], n});
        }
    }
    rep(i,0,N) pi[i] = amt[i] ? pi[i] + dist[i] : INFC;
  }

  pair<F, R> maxflow(int s, int t) { // 2126d0
    F totflow = 0; R totcost = 0;
    while (path(s), amt[t]) {
      F fl = amt[t]; totflow += fl;
      for(int n = t;n != s;) {
        auto &[p, f, c, rev] = ed[n][par[n]];
        f -= fl; ed[p][rev].flow += fl; n = p;
        totcost -= (R) fl * c;//OR +=(R)fl*ed[p][rev].cost
      }
    }
    return {totflow, totcost};
  }

  // If some costs can be negative, call this before maxflow
     :
  void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INFC); pi[s] = 0;
    int it = N, ch = 1;
    while (ch-- && it--)
      rep(i,0,N) if (pi[i] != INFC)
        for (auto [to, f, c, _]: ed[i])
          if (f < 0 && ckmin(pi[to], pi[i] + c)) ch = 1;
    assert(it >= 0); // negative cost cycle
  }
}; // 1c3c23 without setpi; 4f8c64 with setpi
```

NetworkSimplex.h

Description: Network simplex algorithm

Time: empircally fast ae1af8, 69 lines

```
using i128 = __int128_t;
struct NetworkSimplex {
  using Flow = int64_t; using Cost = int64_t; using V_id =
     int32_t; using E_id = int32_t;
  struct Edge { V_id src, dst; Flow flow, cap; Cost cost; };
  Cost INF = 1; i128 delta = 0;
  int n; vector<Flow> B; vector<Cost> P; vector<Edge> E;
  vi pei, depth; vector<set<int>> tree;
  void init(int _n) { n = _n; B.resize(n+1); pei.assign(n+1
     ,-1);
    depth.resize(n+1); P.resize(n+1); tree.resize(n+1); }
  int ae(V_id a, V_id b, Flow l, Flow u, Cost c) {
    E.pb({a,b,0,u-l,c}); E.pb({b,a,0,0,-c});
    delta += i128(l)*c; B[b] += l, B[a] -= l;
    return sz(E)-2;
  }
  void upd(E_id ei) {
    P[E[ei].dst] = P[E[ei].src]+E[ei].cost;
    pei[E[ei].dst] = ei^1;
    depth[E[ei].dst] = 1+depth[E[ei].src];
    dfs(E[ei].dst);
  }
  void dfs(int node) { for(auto& ei:tree[node]) if (ei !=
     pei[node]) upd(ei); }
  // applies cb to a -> b and (tree path b -> a)
  template<class CB> void walk(int ei, CB cb) {
    cb(ei, 0);
    for (V_id a = E[ei].src, b = E[ei].dst; a != b; ) {
      if (depth[a] > depth[b]) cb(pei[a]^1, 0), a = E[pei[a
     ]].dst;
      else cb(pei[b], 1), b = E[pei[b]].dst;
    }
  }
  i128 solve() {
    const int m = sz(E);
    for (E_id i = 0; i < m; i += 2) INF += abs(E[i].cost);
    rep(i,0,n) {
      V_id a = n, b = i; Cost c = B[i];
      if (c < 0) c *= -1, swap(a,b);
      E_id ei = ae(a,b,0,c,-INF);
      tree[a].insert(ei), tree[b].insert(ei^1);
    }
    dfs(n);
    i128 answer = delta;
    E_id ptr = 0;
    const int BLOCK = n/3+1;
    rep(z,0,sz(E)/BLOCK+1) {
      pair<Cost,E_id> pin{0,-1};
      for (int t = 0; t < BLOCK; ++t, (++ptr) %= sz(E)) {
        const auto& e = E[ptr];
        if (e.flow < e.cap) ckmin(pin, mp(e.cost+P[e.src]-P[
     e.dst],ptr));
      }
      auto [cost, ein] = pin;
      if (cost == 0) continue;
      pair<Cost,E_id> pout{E[ein].cap-E[ein].flow, ein};
      walk(ein,[&](E_id ei, bool late) { // last blocking
     arc, avoids stalling
        Flow r = E[ei].cap-E[ei].flow;
        if (r < pout.first || (late && r == pout.first))
     pout = {r,ei}; });
      auto [flow, eout] = pout;
      walk(ein,[&](E_id ei, bool) { E[ei].flow += flow, E[ei
     ^1].flow -= flow; });
```

```
      tree[E[ein].src].insert(ein), tree[E[ein].dst].insert(ein^1);
      tree[E[eout].src].erase(eout), tree[E[eout].dst].erase(eout^1);
      upd(pei[E[eout].src] == eout ? ein : ein^1);
      answer += i128(flow)*cost; // why can't this loop?
      z = -1;
    }
    rep(i,0,n) {
      if (E[m+2*i].flow < E[m+2*i].cap) throw 5;
      answer += i128(E[m+2*i].flow)*INF;
    }
    return answer;
  }
};
```

Dinic.h

Description: Dinic's without scaling. Self-loops are ignored; requires $S \neq T$. The i-th added non-loop edge is E[2*i], with its flow in E[2*i].flow.
Usage: `Dinic d(n); d.AddEdge(u, v, cap); ll f = d.MaxFlow(s, t);`
93b568, 64 lines

```
struct Dinic {
  struct Edge {
    int u, v;
    ll cap, flow;
    Edge() {}
    Edge(int u, int v, ll cap): u(u), v(v), cap(cap), flow(0) {}
  };
  int N;
  vector<Edge> E;
  vector<vector<int>> g;
  vector<int> d, pt;
  Dinic(int N): N(N), E(0), g(N), d(N), pt(N) {}
  void AddEdge(int u, int v, ll cap) {
    if (u != v) {
      E.emplace_back(u, v, cap);
      g[u].emplace_back(E.size() - 1);
      E.emplace_back(v, u, 0);
      g[v].emplace_back(E.size() - 1);
    }
  }
  bool BFS(int S, int T) {
    queue<int> q({S});
    fill(d.begin(), d.end(), N + 1);
    d[S] = 0;
    while(!q.empty()) {
      int u = q.front(); q.pop();
      if (u == T) break;
      for (int k: g[u]) {
        Edge &e = E[k];
        if (e.flow < e.cap && d[e.v] > d[e.u] + 1) {
          d[e.v] = d[e.u] + 1;
          q.emplace(e.v);
        }
      }
    }
    return d[T] != N + 1;
  }
  ll DFS(int u, int T, ll flow = -1) {
    if (u == T || flow == 0) return flow;
    for (int &i = pt[u]; i < g[u].size(); ++i) {
      Edge &e = E[g[u][i]];
      Edge &oe = E[g[u][i]^1];
      if (d[e.v] == d[e.u] + 1) {
        ll amt = e.cap - e.flow;
        if (flow != -1 && amt > flow) amt = flow;
        if (ll pushed = DFS(e.v, T, amt)) {
          e.flow += pushed;
          oe.flow -= pushed;
          return pushed;
        }
      }
    }
    return 0;
  }
  ll MaxFlow(int S, int T) {
    ll total = 0;
    while (BFS(S, T)) {
      fill(pt.begin(), pt.end(), 0);
      while (ll flow = DFS(S, T))
        total += flow;
    }
    return total;
  }
};
```

DinicWithScaling.h

Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.
d7f0f1, 42 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s, only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time: $\mathcal{O}\left(V^3\right)$
8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i};
  rep(ph,1,n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();
      rep(i,0,n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations
"PushRelabel.h" 0418b3, 13 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vi par(N);
  rep(i,1,N) {
    PushRelabel D(N);
    for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
    tree.push_back({i, par[i], D.calc(i, par[i])});
    rep(j,i+1,N)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
  }
  return tree;
}
```

7.2 Matching

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vi btoa(m, -1); dfsMatching(g, btoa);`
Time: $\mathcal{O}(VE)$
522b98, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
  if (btoa[j] == -1) return 1;
  vis[j] = 1; int di = btoa[j];
  for (int e : g[di])
    if (!vis[e] && find(e, g, btoa, vis)) {
      btoa[e] = di;
      return 1;
    }
  return 0;
}
int dfsMatching(vector<vi>& g, vi& btoa) {
  vi vis;
  rep(i,0,sz(g)) {
    vis.assign(sz(btoa), 0);
    for (int j : g[i])
      if (find(j, g, btoa, vis)) {
        btoa[j] = i;
        break;
      }
  }
  return sz(btoa) - (int)count(all(btoa), -1);
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.
"DFSMatching.h" da4196, 20 lines

```
vi cover(vector<vi>& g, int n, int m) {
  vi match(m, -1);
  int res = dfsMatching(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vi q, cover;
  rep(i,0,n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {
      seen[e] = true;
      q.push_back(match[e]);
    }
  }
  rep(i,0,n) if (!lfound[i]) cover.push_back(i);
  rep(i,0,m) if (seen[i]) cover.push_back(n+i);
  assert(sz(cover) == res);
```

```
  return cover;
}
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.

Time: $\mathcal{O}\left(N^2M\right)$ ba10e5, 28 lines

```
template<class T>
pair<T, vi> hungarian(const vector<vector<T>>& C) {
  int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
  vector<T> dist(m), pot(m);
  vi match(i), rev(m, -1), cols(m), prev(m);
  T d = 0, nd, cost = 0;
  while (i--) {
    rep(c,0,m) dist[c] = C[i][c], cols[c] = c, prev[c] = i;
    for (s = 0;;) {
      rep(j,s,m) {
        c = cols[j], nd = dist[c] - pot[c];
        if (j == s || d > nd) d = nd, swap(cols[s], cols[j]);
      }
      if ((r = rev[c = cols[s++]]) == -1) break;
      rep(j,0,m) if (dist[j] > (nd = C[r][j]-C[r][c]+dist[c])) dist[j] = nd, prev[j] = r;
    }
    cost += dist[c];
    while (s--) pot[cols[s]] = dist[cols[s]] - d;
    for (; r != i; swap(c, match[r]))
      r = rev[c] = prev[c];
  }
  return {cost, match};
}

pair<int, vi> hungarian(const vector<vi>& C) {
  return hungarian<int>(C);
}
```

GeneralMatchingBlossom.h

Description: Variant on Gabow's Impl of Edmond's Blossom Algorithm. General unweighted max matching with 1-based indexing. If `white[v] = 0` after `solve()` returns, `v` is part of every max matching. 1-INDEXED NODES

Time: $\mathcal{O}(NM)$, faster in practice 6dfcb8, 53 lines

```
using vb = vector<bool>;
using pi = pair<int,int>;
using vpi = vector<pi>;
struct MaxMatching {
  int N; vector<vi> adj;
  vector<int> mate, first; vb white; vpi label;
  void init(int _N) { N = _N; adj = vector<vi>(N+1);
    mate = first = vi(N+1); label = vpi(N+1); white = vb(N+1); }
  void ae(int u, int v) { adj.at(u).pb(v), adj.at(v).pb(u); }
  int group(int x) { if (white[first[x]]) first[x] = group(first[x]);
    return first[x]; }
  void match(int p, int b) {
    swap(b,mate[p]); if (mate[b] != p) return;
    if (!label[p].second) mate[b] = label[p].first, match(label[p].first,b); // vertex label
    else match(label[p].first,label[p].second), match(label[p].second,label[p].first); // edge label
  }
  bool augment(int st) { assert(st);
    white[st] = 1; first[st] = 0; label[st] = {0,0};
    queue<int> q; q.push(st);
    while (!q.empty()) {
      int a = q.front(); q.pop(); // outer vertex
      for(auto &b:adj[a]) { assert(b);
        if (white[b]) { // two outer vertices, form blossom
          int x = group(a), y = group(b), lca = 0;
          while (x||y) {
            if (y) swap(x,y);
            if (label[x] == pi{a,b}) { lca = x; break; }
            label[x] = {a,b}; x = group(label[mate[x]].first);
          }
          for (int v: {group(a),group(b)}) while (v != lca) {
            assert(!white[v]); // make everything along path white
            q.push(v); white[v] = true; first[v] = lca;
            v = group(label[mate[v]].first);
          }
        } else if (!mate[b]) { // found augmenting path
          mate[b] = a; match(a,b); white = vb(N+1); // reset
          return true;
        } else if (!white[mate[b]]) {
          white[mate[b]] = true; first[mate[b]] = b;
          label[b] = {0,0}; label[mate[b]] = pi{a,0};
          q.push(mate[b]);
        }
      }
    }
    return false;
  }
  int solve() {
    int ans = 0; // mate pairs node with its match, bidirectional
    rep(st,1,N+1) if (!mate[st]) ans += augment(st);
    rep(st,1,N+1) if (!mate[st] && !white[st]) assert(!augment(st));
    return ans;
  }
};
```

GeneralWeightedMatching.h

Description: General max weight matching with 1-based indexing (weight is maximized, NOT cardinality). Edge weights must be positive, combo of UnweightedMatch and Hungarian. 1-INDEXED NODES

Time: $\mathcal{O}\left(N^3\right)$? fe0333, 145 lines

```
template<int SZ> struct WeightedMatch {
  struct edge { int u,v,w; }; edge g[SZ*2][SZ*2];
  void ae(int u, int v, int w) { g[u][v].w = g[v][u].w = w; }
  int N,NX,lab[SZ*2],match[SZ*2],slack[SZ*2],st[SZ*2];
  int par[SZ*2],floFrom[SZ*2][SZ],S[SZ*2],aux[SZ*2];
  vi flo[SZ*2]; queue<int> q;
  void init(int _N) { N = _N; // init all edges
    rep(u,1,N+1) rep(v,1,N+1) g[u][v] = {u,v,0}; }
  int eDelta(edge e) { // >= 0 at all times
    return lab[e.u]+lab[e.v]-g[e.u][e.v].w*2; }
  void updSlack(int u, int x) { // smallest edge -> blossom x
    if (!slack[x] || eDelta(g[u][x]) < eDelta(g[slack[x]][x]))
      slack[x] = u; }
  void setSlack(int x) {
    slack[x] = 0; rep(u,1,N+1) if (g[u][x].w > 0
     && st[u] != x && S[st[u]] == 0) updSlack(u,x); }
  void qPush(int x) {
    if (x <= N) q.push(x);
    else for(auto &t:flo[x]) qPush(t); }
  void setSt(int x, int b) {
    st[x] = b; if (x > N) for(auto &t:flo[x]) setSt(t,b); }
  int getPr(int b, int xr) { // get even position of xr
    int pr = find(all(flo[b]),xr)-begin(flo[b]);
    if (pr&1) { reverse(1+all(flo[b])); return sz(flo[b])-pr; }
    return pr; }
  void setMatch(int u, int v) { // rearrange flo[u], matches
    edge e = g[u][v]; match[u] = e.v; if (u <= N) return;
    int xr = floFrom[u][e.u], pr = getPr(u,xr);
    rep(i,0,pr) setMatch(flo[u][i],flo[u][i^1]);
    setMatch(xr,v); rotate(begin(flo[u]),pr+all(flo[u])); }
  void augment(int u, int v) { // set matches including u->v
    while (1) { // and previous ones
      int xnv = st[match[u]]; setMatch(u,v);
      if (!xnv) return;
      setMatch(xnv,st[par[xnv]]);
      u = st[par[xnv]], v = xnv;
    }
  }
  int lca(int u, int v) { // same as in unweighted
    static int t = 0; // except maybe return 0
    for (++t;u||v;swap(u,v)) {
      if (!u) continue;
      if (aux[u] == t) return u;
      aux[u] = t; u = st[match[u]];
      if (u) u = st[par[u]];
    }
    return 0;
  }
  void addBlossom(int u, int anc, int v) {
    int b = N+1; while (b <= NX && st[b]) ++b;
    if (b > NX) ++NX; // new blossom
    lab[b] = S[b] = 0; match[b] = match[anc]; flo[b] = {anc};
    auto blossom = [&](int x) {
      for (int y; x != anc; x = st[par[y]])
        flo[b].pb(x), flo[b].pb(y = st[match[x]]), qPush(y);
    };
    blossom(u); reverse(1+all(flo[b])); blossom(v); setSt(b,b);
    // identify all nodes in current blossom
    rep(x,1,NX+1) g[b][x].w = g[x][b].w = 0;
    rep(x,1,N+1) floFrom[b][x] = 0;
    for(auto &xs:flo[b]) { // find tightest constraints
      rep(x,1,NX+1) if (g[b][x].w == 0 || eDelta(g[xs][x]) <
        eDelta(g[b][x])) g[b][x]=g[xs][x], g[x][b]=g[x][xs];
      rep(x,1,N+1) if (floFrom[xs][x]) floFrom[b][x] = xs;
    } // floFrom to deconstruct blossom
    setSlack(b); // since didn't qPush everything
  }
  void expandBlossom(int b) {
    for(auto &t:flo[b]) setSt(t,t); // undo setSt(b,b)
    int xr = floFrom[b][g[b][par[b]].u], pr = getPr(b,xr);
    for(int i = 0; i < pr; i += 2) {
      int xs = flo[b][i], xns = flo[b][i+1];
      par[xs] = g[xns][xs].u; S[xs] = 1; // no setSlack(xns)?
      S[xns] = slack[xs] = slack[xns] = 0; qPush(xns);
    }
    S[xr] = 1, par[xr] = par[b];
    rep(i,pr+1,sz(flo[b])) { // matches don't change
      int xs = flo[b][i]; S[xs] = -1, setSlack(xs); }
    st[b] = 0; // blossom killed
  }
  bool onFoundEdge(edge e) {
    int u = st[e.u], v = st[e.v];
    if (S[v] == -1) { // v unvisited, matched with smth else
      par[v] = e.u, S[v] = 1; slack[v] = 0;
      int nu = st[match[v]]; S[nu] = slack[nu] = 0; qPush(nu);
    } else if (S[v] == 0) {
      int anc = lca(u,v); // if 0 then match found!
      if (!anc) return augment(u,v),augment(v,u),1;
      addBlossom(u,anc,v);
    }
    return 0;
  }
  bool matching() {
    q = queue<int>();
    rep(x,1,NX+1) {
      S[x] = -1, slack[x] = 0; // all initially unvisited
      if (st[x] == x && !match[x]) par[x] = S[x] = 0, qPush(x);
    }
    if (!sz(q)) return 0;
    while (1) {
      while (sz(q)) { // unweighted matching with tight edges
        int u = q.front(); q.pop(); if (S[st[u]] == 1) continue;
        rep(v,1,N+1) if (g[u][v].w > 0 && st[u] != st[v]) {
          if (eDelta(g[u][v]) == 0) { // condition is strict
            if (onFoundEdge(g[u][v])) return 1;
          } else updSlack(u,st[v]);
        }
      }
      int d = INT_MAX;
      rep(b,N+1,NX+1) if (st[b] == b && S[b] == 1)
        ckmin(d,lab[b]/2); // decrease lab[b]
      rep(x,1,NX+1) if (st[x] == x && slack[x]) {
        if (S[x] == -1) ckmin(d,eDelta(g[slack[x]][x]));
        else if (S[x] == 0) ckmin(d,eDelta(g[slack[x]][x])/2);
      } // edge weights shouldn't go below 0
      rep(u,1,N+1) {
        if (S[st[u]] == 0) {
```

```
          if (lab[u] <= d) return 0; // why?
          lab[u] -= d;
        } else if (S[st[u]] == 1) lab[u] += d;
      } // lab has opposite meaning for verts and blossoms
      rep(b,N+1,NX+1) if (st[b] == b && S[b] != -1)
        lab[b] += (S[b] == 0 ? 1 : -1)*d*2;
      q = queue<int>();
      rep(x,1,NX+1) if (st[x]==x && slack[x] // new tight edge
          && st[slack[x]] != x && eDelta(g[slack[x]][x]) == 0)
        if (onFoundEdge(g[slack[x]][x])) return 1;
      rep(b,N+1,NX+1) if (st[b]==b && S[b]==1 && lab[b]==0)
        expandBlossom(b); // odd dist blossom taken apart
    }
    return 0;
  }
  pair<ll,int> calc() {
    NX = N; st[0] = 0; rep(i,1,2*N+1) aux[i] = 0;
    rep(i,1,N+1) match[i] = 0, st[i] = i, flo[i].clear();
    int wMax = 0;
    rep(u,1,N+1) rep(v,1,N+1)
      floFrom[u][v] = (u == v ? u : 0), ckmax(wMax,g[u][v].w);
    rep(u,1,N+1) lab[u] = wMax; // start high and decrease
    int num = 0; ll wei = 0; while (matching()) ++num;
    rep(u,1,N+1) if (match[u] && match[u] < u)
      wei += g[u][match[u]].w; // edges in matching
    return {wei,num};
  }
};
```

7.3 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.

Usage: `scc(graph, [&](vi& v) { ... })` visits all components in reverse topological order. `comp[i]` holds the component index of a node (a component only has edges to components with lower index). `ncomps` will contain the number of components.

Time: $\mathcal{O}(E+V)$

76b5c9, 24 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont); cont.clear();
    ncomps++;
  }
  return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two distinct paths between any two nodes. Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.

Usage: `int eid = 0; ed.resize(N);`
`for each edge (a,b) {`
`ed[a].emplace_back(b, eid);`
`ed[b].emplace_back(a, eid++); }`
`bicomps([&](const vi& edgelist) {...});`

Time: $\mathcal{O}(E+V)$

2965e5, 33 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
  int me = num[at] = ++Time, e, y, top = me;
  for (auto pa : ed[at]) if (pa.second != par) {
    tie(y, e) = pa;
    if (num[y]) {
      top = min(top, num[y]);
      if (num[y] < me)
        st.push_back(e);
    } else {
      int si = sz(st);
      int up = dfs(y, e, f);
      top = min(top, up);
      if (up == me) {
        st.push_back(e);
        f(vi(st.begin() + si, st.end()));
        st.resize(si);
      }
      else if (up < me) st.push_back(e);
      else { /* e is a bridge */ }
    }
  }
  return top;
}

template<class F>
void bicomps(F f) {
  num.assign(sz(ed), 0);
  rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

BlockCutTree.h

Description: Builds the block cut tree. BCTree node $n >= cut$ is the single node who[n][0] of the original graph: an AP, an endpoint of a bridge (bridges join two such nodes directly) or an isolated node. Node v of the original graph is such a node iff $vmap[v] >= cut$. No self-loops. $emap[i] = -1$ if edge i is a bridge. Otherwise, $emap[i]$ is the BCC containing it.

Usage: see BiconnectedComponents.h
`edges[i] = the edge i (pair of two nodes)`

Time: $\mathcal{O}(E+V)$

"BiconnectedComponents.h" ea2e5d, 49 lines

```
vector<pii> edges;
tuple<int, vector<vi>, vector<vi>, vi, vi> BCTree() {
  int N = ed.size(), M = edges.size();
  vector<int> emap(M, -1); // edge -> bicomp id
  vector<vi> bclist; // list of biconnected components
  bicomps([&](vector<int> &&eds) {
    for(int x: eds) emap[x] = bclist.size();
    bclist.emplace_back(eds);
  });

  vector<int> vmap(N, -1);
  for(int i = 0;i < M; ++i)
    if(emap[i] == -1) { // bridge: connects two APs
      auto [u, v] = edges[i];
      vmap[u] = vmap[v] = -2;
    }

  for(int i = 0;i < bclist.size();++i)
    for(int x: bclist[i]) {
      auto [u, v] = edges[x];
      for(int j = 2;j--; swap(u, v))
        if(vmap[u] == -1) vmap[u] = i;
        else if(vmap[u] != i) vmap[u] = -2;
    }

  int const cut = bclist.size();
  int TN = bclist.size();
  vector<vi> who(TN);
  for(int i = 0;i < N;++i)
    if(vmap[i] < 0) vmap[i] = TN++, who.emplace_back(1, i);
    else who[vmap[i]].emplace_back(i);
  vector<vi> tadj(TN);

  for(int i = 0;i < N; ++i)
    if(cut <= vmap[i]) // if `i` is an AP
      for(auto [x, e]: ed[i]) {
        if(emap[e] == -1) // Bridge: connect both APs
          tadj[vmap[i]].push_back(vmap[x]);
        else {
          tadj[vmap[i]].push_back(emap[e]);
          tadj[emap[e]].push_back(vmap[i]);
        }
      }
  for(auto &v: tadj) { // one AP can connect to a BCC in multiple ways
    sort(all(v));
    v.resize(distance(v.begin(), unique(all(v))));
  }
  return {cut, tadj, who, emap, vmap};
}
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&\ldots$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (`~x`).

Usage: `TwoSat ts(number of boolean variables);`
`ts.either(0, ~3); // Var 0 is true or var 3 is false`
`ts.setValue(2); // Var 2 is true`
`ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true`
`ts.solve(); // Returns true iff it is solvable`
`ts.values[0..N-1] holds the assigned values to the vars`

Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.

5f9706, 56 lines

```
struct TwoSat {
  int N;
  vector<vi> gr;
  vi values; // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), gr(2*n) {}

  int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x) { either(x, x); }

  void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
  }

  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
  }
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V+E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
    if (it == end){ ret.push_back(x); s.pop_back(); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
  return {ret.rbegin(), ret.rend()};
}
```

BipolarOrientation.h

Description: Finds a bipolar orientation of a biconnected graph
Time: $\mathcal{O}(M)$

adc2bf, 35 lines

```
// `a` is taken by value: the caller's graph is not modified
vector<int> bipolarOrient(vector<vector<int> > a, int s, int t) {
  size_t N = a.size(); // must have s != t, N >= 2
  vector<int> o(N), p(N, -1), d(N, -1), l(N), lk[2];
  lk[0]=lk[1]=vector<int>(N,-1); // lk[0] = prev, lk[1] = next
  a[s].insert(a[s].begin(), t); // can duplicate edge
  int time=0;
  auto f=[&](auto& f, int n) ->void{
    o[time]=n,l[n]=d[n]=time++;
    for(int x:a[n]) if(x!=p[n])
    {
      if(d[x]==-1) {
        p[x]=n, f(f, x); // assert(n==s || l[x]<d[n]);
        ckmin(l[n], l[x]);
      } else ckmin(l[n], d[x]);
    }
  };
  f(f, s);
  auto add=[&](int u, int v, bool b){
    lk[!b][v]=lk[!b][u]; // b true: before, b false: after
    lk[!b][u]=v;
    lk[b][v]=u;
    if(lk[!b][v]!=-1) lk[b][lk[!b][v]]=v;
  };
  add(s, t, 0);
  vector<char> sgn(N, 0);
  sgn[t]=1;
  for(int i=2;i<N;++i) {
    int n=o[i];
    add(p[n], n, sgn[p[n]]=!sgn[o[l[n]]]);
  } // assert(lk[0][s] == -1);
  vector<int> ans;
  for(;s!=-1;s=lk[1][s]) ans.push_back(s);
  return ans;
}
```

7.4 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
  vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
  for (pii e : eds) ++cc[e.first], ++cc[e.second];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vector<vi> adj(N, vi(ncols, -1));
  for (pii e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
      int left = fan[i], right = fan[++i], e = cc[i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
      free[right] = e;
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);
  }
  rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
  return ret;
}
```

7.5 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.
Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
  if (!P.any()) { if (!X.any()) f(R); return; }
  auto q = (P | X)._Find_first();
  auto cands = P & ~eds[q];
  rep(i,0,sz(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
  }
}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.
Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  }
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        }
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
  }
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.

7.6 Trees

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" 0f62fb, 21 lines

```
struct LCA {
  int T = 0;
  vi time, path, ret;
  RMQ<int> rmq;

  LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
  void dfs(vector<vi>& C, int v, int par) {
    time[v] = T++;
    for (int y : C[v]) if (y != par) {
      path.push_back(v), ret.push_back(time[v]);
      dfs(C, y, v);
    }
  }

  int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
  //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S|-1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time: $\mathcal{O}(|S| \log |S|)$

"LCA.h" 9775a0, 21 lines

```
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
  static vi rev; rev.resize(sz(lca.time));
  vi li = subset, &T = lca.time;
  auto cmp = [&](int a, int b) { return T[a] < T[b]; };
  sort(all(li), cmp);
  int m = sz(li)-1;
  rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
```

```
  }
  sort(all(li), cmp);
  li.erase(unique(all(li)), li.end());
  rep(i,0,sz(li)) rev[li[i]] = i;
  vpi ret = {pii(0, li[0])};
  rep(i,0,sz(li)-1) {
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  }
  return ret;
}
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.

Time: $\mathcal{O}\left((\log N)^2\right)$

"../data-structures/LazySegmentTree.h" 6f34db, 46 lines

```
template <bool VALS_EDGES> struct HLD {
  int N, tim = 0;
  vector<vi> adj;
  vi par, siz, depth, rt, pos;
  Node *tree;
  HLD(vector<vi> adj_)
    : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1), depth(N),
      rt(N),pos(N),tree(new Node(0, N)){ dfsSz(0); dfsHld(0); }
  void dfsSz(int v) {
    if (par[v] != -1) adj[v].erase(find(all(adj[v]), par[v]));
    for (int& u : adj[v]) {
      par[u] = v, depth[u] = depth[v] + 1;
      dfsSz(u);
      siz[v] += siz[u];
      if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    }
  }
  void dfsHld(int v) {
    pos[v] = tim++;
    for (int u : adj[v]) {
      rt[u] = (u == adj[v][0] ? rt[v] : u);
      dfsHld(u);
    }
  }
  template <class B> void process(int u, int v, B op) {
    for (; rt[u] != rt[v]; v = par[rt[v]]) {
      if (depth[rt[u]] > depth[rt[v]]) swap(u, v);
      op(pos[rt[v]], pos[v] + 1);
    }
    if (depth[u] > depth[v]) swap(u, v);
    op(pos[u] + VALS_EDGES, pos[v] + 1);
  }
  void modifyPath(int u, int v, int val) {
    process(u, v, [&](int l, int r) { tree->add(l, r, val); });
  }
  int queryPath(int u, int v) { // Modify depending on problem
    int res = -1e9;
    process(u, v, [&](int l, int r) {
      res = max(res, tree->query(l, r));
    });
    return res;
  }
  int querySubtree(int v) { // modifySubtree is similar
    return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
  }
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.

Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 39e620, 60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```

Centroid.h

Description: Boilerplate centroid decomp code

Time: $\mathcal{O}(N \log N)$

db481b, 32 lines

```
struct Centroid {
  vector<vi> const& adj;
  int N;
  vector<int> s, rem;
  vector<vi> links;
  vector<pair<int, int> > par; // <parent centroid, index>
  int root;
  int dfs(int n, int p=-1) {
    s[n]=1;
    for(int x: adj[n]) if(x!=p && !rem[x])
      s[n]+=dfs(x,n);
    return s[n];
  }
  int find(int n, int t, int p=-1) {
    for(int k=1;k--;)
      for(int x:adj[n]) if(x!=p && !rem[x] && s[x]*2>t)
      {p=n,n=x,k=1; break;}
    return n;
  }
  int cent(int start=0) {
    int c = find(start, dfs(start));
    // Do stuff with c. Just remember to check both (x != p && !rem[x])
    rem[c]=1;
    for(int x:adj[c]) if(!rem[x]) {
      int v = cent(x);
      par[v] = {c, sz(links[c])};
      links[c].push_back(v);
    }
    return c;
  }
  Centroid(vector<vi> const& adj): adj(adj), N(adj.size()),
    s(N), rem(N), links(N), par(N,{-1,-1}), root(cent()) {}
};
```

JacobLinkCut.h

Description: Link-cut tree with evert, node update and path sum.

Time: All operations take amortized $\mathcal{O}(\log N)$.

ae6b6c, 80 lines

```
struct node {
  node *p,*c[2];
  ll v, subv;
  int rev;
  node():p(NULL),rev(0),v(0),subv(0){c[0]=c[1]=NULL;}
  node(node *_p):p(_p),rev(0),v(0),subv(0){c[0]=c[1]=NULL;}
  int state() {
    if (!p) return -1;
    if (this==p->c[0]) return 0;
    else if (this==p->c[1]) return 1;
    return -1;
  }
  bool isroot() { return state()==-1; }
  node* prop() { // propagate rev, pushdown
    if (rev) {
      rev=0;
      swap(c[0],c[1]);
      if (c[0]) c[0]->rev^=1;
      if (c[1]) c[1]->rev^=1;
    }
    return this;
  }
  void set() { // update any subtrees, pullup
    subv=v;
    if (c[0]) subv+=c[0]->subv;
    if (c[1]) subv+=c[1]->subv;
  }
  void res() {
    if (p->p) p->p->prop();
    p->prop(); prop();
  }
  friend void linknode(node *px, node *x, int d) {
    if (px && d!=-1) px->c[d] = x;
    if (x) x->p = px;
  }
  void rot() {
    int d=state(),d2=p->state();
    node *b=c[!d],*pp=p,*gp=p->p;
    linknode(pp,b,d);
    linknode(this,pp,!d);
    linknode(gp,this,d2);
    c[!d]->set(); set();
  }
  node* splay() {
    while(!isroot()) {
      res();
      if (p->isroot()) rot();
      else if (state()==p->state()) p->rot(),rot();
      else rot(),rot();
    }
    return prop();
  }
  node* find_min() {
    node *x=this;
    while(x->prop()->c[0]) x=x->c[0];
    return x->splay();
  }
};
void access(node *x) {
  node *prev=NULL;
  for (node *z=x;z;z=z->p) {
    z->splay();
```

```
    z->c[1]=prev;
    z->set();
    prev=z;
  }
  x->splay();
}
void evert(node *x) {access(x); x->rev^=1;}
void link(node *x,node *y) {
  evert(x); access(x); access(y);
  y->c[1]=x; x->p=y; y->set();
}
void cut(node *x,node *y) {
  evert(y); access(x); access(y);
  x->p=NULL;
}
node* find_root(node *x) {access(x); return x->find_min();}
void update(node *x, ll v) {access(x); x->v += v; x->splay()
    ;}
ll query(node *x, node *y) {evert(x); access(y); return y->
    subv;}
```

Geometry (8)

8.1 Geometric primitives

Point.h

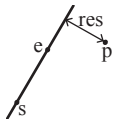
Description: Class to handle points in the plane. T can be e.g. double, long long or Fraction. (Avoid int.) Fraction supports rational operations; unit/normal/rotate require a floating-point type. dist/angle return approximate doubles.

22bff9, 28 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0);
    }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y)
    ; }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y)
    ; }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this);
    }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2((double)y, (double)x);
    }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

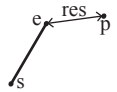


"Point.h" f6bf6b, 4 lines

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

Description:
Returns the shortest distance between point p and the line segment from point s to e.



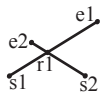
Usage:
```
Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;
```

"Point.h" d5f39f, 6 lines

```
typedef Point<double> P;
double segDist(P s, P e, P p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. For integer coordinates with absolute value at most 10^9, classification is exact using 128-bit intermediates. The output defaults to Point<long double>; choose segInter<T> for another floating type or Fraction. Fraction output throws if a reduced coordinate does not fit. Rounding never changes the vector size, even if overlap endpoints round to equal points. Floating-point input can misclassify nearly collinear segments. Fraction input is exact only while every intermediate fits. These inputs retain their scalar type. Requires C++17.



Usage:
```
auto inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
```

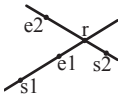
"Point.h", "OnSegment.h", "../number-theory/Fraction.h" 4e6580, 44 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
  static_assert(!numeric_limits<decltype(a.x)>::is_integer,
    "Choose a floating scalar or Fraction for integer input"
    );
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}

template<class T = long double, class I, enable_if_t<
  numeric_limits<I>::is_integer && (is_floating_point<T>::
    value ||
  is_same<T, __float128>::value || is_same<T, Fraction>::
    value), int> = 0>
vector<Point<T>> segInter(Point<I> a, Point<I> b, Point<I> c
    , Point<I> d) {
  using W = __int128_t;
  using P = Point<W>;
  auto wide = [](Point<I> p) { return P(p.x,p.y); };
  P A=wide(a), B=wide(b), C=wide(c), D=wide(d);
  auto ratio = [](W n, W den) -> T {
    if constexpr (is_same<T, Fraction>::value)
      return Fraction::fromWide(n,den);
    else return T(n)/T(den);
  };
  auto point = [&](P p, W den) {
    return Point<T>(ratio(p.x,den),ratio(p.y,den));
  };
  W oa=C.cross(D,A), ob=C.cross(D,B), oc=A.cross(B,C), od=A.
    cross(B,D);
  if (sgn(oa)*sgn(ob)<0 && sgn(oc)*sgn(od)<0)
    return {point(A*ob-B*oa,ob-oa)};
  set<P> s;
  if (onSegment(C,D,A)) s.insert(A);
  if (onSegment(C,D,B)) s.insert(B);
  if (onSegment(A,B,C)) s.insert(C);
  if (onSegment(A,B,D)) s.insert(D);
  vector<Point<T>> out;
  for (P p : s) out.push_back(point(p,1));
  return out;
}
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage:
```
auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
```

"Point.h" a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

LineIntersectionArea.h

Description: Signed doubled area of (a, c, X), where X is the intersection of the nonparallel lines ab and cd. Integer coordinates must have absolute value at most 10^9. Computes UV/D, where $U = (c-a)\times(b-a)$, $V = (c-a)\times(d-c)$ and $D = (b-a)\times(d-c)$. Integer intermediates are exact; the result is approximate. Use T=__float128 when later area sums may cancel.

Usage:
```
auto area2 = lineInterArea2<__float128>(a,b,c,d);
```

Time: $\mathcal{O}(1)$

"Point.h" 17fc3c, 17 lines

```
template<class T = long double, class I>
T lineInterArea2(Point<I> a, Point<I> b,
    Point<I> c, Point<I> d) {
  static_assert(numeric_limits<I>::is_integer,
    "lineInterArea2 requires integer coordinates");
  static_assert(is_floating_point<T>::value ||
    is_same<T, __float128>::value,
    "lineInterArea2 requires a floating-point result");
  auto wide = [](Point<I> p) {
    return Point<__int128_t>(p.x, p.y);
  };
  auto A=wide(a), B=wide(b), C=wide(c), D=wide(d);
  auto u=A.cross(C,B), v=D.cross(A,C);
  auto den=(B-A).cross(D-C);
  assert(den != 0);
  return T(u*v) / T(den);
}
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage:
```
bool left = sideOf(p1,p2,q)==1;
```

"Point.h" 3af81c, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

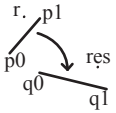
OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h



Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

"Point.h" 8cfadb, 7 lines

```
typedef Point<double> P;
template<class P>
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2
      ();
}
```

LineProjRefl.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab insted. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h" b5562d, 5 lines

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
}
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: `vector<Angle> v = {w[0], w[0].t360() ...}; // sorted`
`int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }`
`// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i`

0f0602, 31 lines

```
struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y, t};
      }
  int half() const { assert(x || y); return y < 0 || (y == 0
      && x < 0); }
  Angle t90() const { return {-y, x, t + (half() && x >= 0)
      }; }
  Angle t180() const { return {-x, -y, t + half()}; }
  Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare distances
  return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
         make_tuple(b.t, b.half(), a.x * (ll)b.y);
}
// Given two points, this calculates the smallest angle
    between
// them, i.e., the angle that covers the defined line
    segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
          make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)
      };
}
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" 84d6d3, 10 lines

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out
    ) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d
      2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d
      2);
  *out = {mid + per, mid - per};
  return true; }
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h" b0153d, 12 lines

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2
    ) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out; }
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h" e0cfba, 7 lines

```
template<class P> vector<P> circleLine(P c, double r, P a, P
    b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h}; }
```

CirclePolyIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$

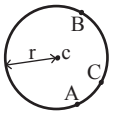
"../../content/geometry/Point.h" 19add1, 18 lines

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2
        ();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det)
        );
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  };
  auto sum = 0.0;
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum; }
```

circumcircle.h

Description:

ccRadius = radius and ccCenter = center of circle through points A, B, C.



"Point.h" f6eb0e, 8 lines

```
typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2; }
template<class P>
P ccCenter(const P& A, const P& B, const P& C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
    }
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Time: expected $\mathcal{O}(n)$

"circumcircle.h" 09dd0a, 17 lines

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}
```

8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0
        ;
  }
  return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon.

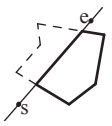
Time: $\mathcal{O}(n)$

"Point.h" 225eb8, 10 lines

```
typedef Point<double> P;
template<class P>
P polygonCenter(const vector<P>& v) {
```

```
  P res(0, 0); decltype(P().x) A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

PolygonCut.h



Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.
Usage: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`

"Point.h" 9ad3b6, 13 lines

```
typedef Point<double> P;
template<class P>
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0) res.push_back(cur);
  }
  return res;
}
```

PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
Time: $\mathcal{O}\left(N^2\right)$, where N is the total number of points

"Point.h", "sideOf.h" 3bdf13, 36 lines

```
typedef Point<double> P;
template<class P>
auto rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
template<class P>
auto polyUnion(vector<vector<P>>& poly) {
  using T = decltype(P().x);
  T ret = 0;
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
    vector<pair<T, int>> segs = {{0, 0}, {1, 0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
        int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
        if (sc != sd) {
          T sa = C.cross(D, A), sb = C.cross(D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
        } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))>0){
          segs.emplace_back(rat(C - A, B - A), 1);
          segs.emplace_back(rat(D - A, B - A), -1);
        }
      }
    }
    sort(all(segs));
    for (auto& s : segs) s.first = min(max(s.first, T(0)), T(1));
    T sum = 0;
    int cnt = segs[0].second;
    rep(j,1,sz(segs)) {
      if (!cnt) sum += segs[j].first - segs[j - 1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }
  return ret / 2;
}
```

ConvexHull.h



Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.
Time: $\mathcal{O}(n \log n)$

"Point.h" 1eb40a, 14 lines

```
typedef Point<ll> P;
template<class P>
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$

"Point.h" 223c2f, 14 lines

```
typedef Point<ll> P;
template<class P>
array<P, 2> hullDiameter(vector<P> S) {
  using T = decltype(P().x);
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<T, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    }
  return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 171dce, 15 lines

```
typedef Point<ll> P;

template<class P>
bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<=-r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet$ $(-1,-1)$ if no collision, $\bullet$ $(i,-1)$ if touching the corner i, $\bullet$ (i,i) if along side $(i,i+1)$, $\bullet$ (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$

"Point.h" 7cf45b, 39 lines

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  }
  return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    }
  return res;
}
```

HalfPlaneIntersection.h

Description: Returns vertices of half-plane intersection. A half-plane is the area to the left of a ray, which is defined by a point `p` and a direction `dp`. Area of intersection should be sufficiently precise when all inputs are integers with magnitude $\leq 10^5$. Intersection must be bounded. Probably works with floating point too (but EPS might need to be adjusted?). Use RayT<Fraction> for exact rational intersections with zero tolerance.
Time: $\mathcal{O}(N \log N)$

"Point.h" eb54e5, 70 lines

```
using P = Point<double>;
using vP = vector<P>;
const double EPS = 1e-9; // adjust as needed

// -1 if lower half, 0 if origin, 1 if upper half, needs to be int type
template<class P>
int half(P x) { return x.y != 0 ? sgn(x.y) : -sgn(x.x); }
template<class P>
bool angleCmp(P a, P b) { int A = half(a), B = half(b);
    return A == B ? a.cross(b) > 0 : A < B; }

template<class T> struct RayT {
  using P = Point<T>;
  P p, dp; // origin, direction
  P isect(const RayT& L) const {
    return p+dp*(L.dp.cross(L.p-p)/L.dp.cross(dp)); }
  bool operator<(const RayT& L) const {
    return angleCmp(dp,L.dp); }
};
using Ray = RayT<double>;

template<class T>
vector<Point<T>> halfPlaneIsect(vector<RayT<T>> rays, bool add_bounds = false) {
  using P = Point<T>;
  using Ray = RayT<T>;
  T eps = 0;
  if constexpr (is_floating_point<T>::value) eps = T(EPS);
  if (add_bounds) { // bound input by rectangle [0,DX] x [0,DY]
    int DX = 1e9, DY = 1e9;
    rays.pb({P{0,0},P{1,0}});
    rays.pb({P{DX,0},P{0,1}});
    rays.pb({P{DX,DY},P{-1,0}});
    rays.pb({P{0,DY},P{0,-1}});
  }
```

```
  sort(all(rays)); // sort rays by angle
  { // remove parallel rays
    vector<Ray> nrays;
    for(auto& t: rays) {
      if (!sz(nrays) || nrays.back().dp.cross(t.dp) > eps) {
      nrays.pb(t); continue; }
      // last two rays are parallel, keep only one
      if (t.dp.cross(t.p-nrays.back().p) > 0) nrays.back() =
      t;
    }
    swap(rays, nrays);
  }
  auto bad = [&](const Ray& a, const Ray& b, const Ray& c) {
    P p1 = a.isect(b), p2 = b.isect(c);
    if ((p2-p1).dot(b.dp) <= eps) {
      if (a.dp.cross(c.dp) <= 0) return 2; // isect(a,b,c) =
      empty
      return 1; // isect(a,c) == isect(a,b,c)
    }
    return 0; // all three rays matter
  };
  #define reduce(t) \
    while (sz(poly) > 1) { \
      int b = bad(poly.at(sz(poly)-2),poly.back(),t); \
      if (b == 2) return {}; \
      if (b == 1) poly.pop_back(); \
      else break; \
    }
  deque<Ray> poly;
    for(auto& t: rays) { reduce(t); poly.pb(t); }
  for(;;poly.pop_front()) {
    reduce(poly[0]);
    if (!bad(poly.back(),poly[0],poly[1])) break;
  }
  assert(sz(poly) >= 3); // expect nonzero area
  vector<P> poly_points; rep(i,0,sz(poly))
    poly_points.pb(poly[i].isect(poly[(i+1)%sz(poly)]));
  return poly_points;
}
```

MinkowskiSum.h

Description: Minkowski sum of two convex polygons given in CCW order. example: diameter of hull A is max in A+(-A)
Time: $\mathcal{O}(N)$

9de734, 19 lines

```
using vP = vector<P>;
vP minkowski_sum(vP a, vP b) {
  if (sz(a) > sz(b)) swap(a, b);
  if (!sz(a)) return {};
  if (sz(a) == 1) {for (auto &t: b) t += a.front(); return b
    ;}
  rotate(begin(a), min_element(all(a)), end(a));
  rotate(begin(b), min_element(all(b)), end(b));
  a.pb(a[0]), a.pb(a[1]);
  b.pb(b[0]), b.pb(b[1]);
  vP result;
  int i = 0, j = 0;
  while (i < sz(a)-2 || j < sz(b)-2) {
    result.pb(a[i]+b[j]);
    T crs = (a[i+1]-a[i]).cross(b[j+1]-b[j]);
    i += (crs >= 0);
    j += (crs <= 0);
  }
  return result;
}
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h" ac41a6, 17 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d
    );
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights w(p, q) = —p.x - q.x— + —p.y - q.y—. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
Time: $\mathcal{O}(N \log N)$

"Point.h" df6f59, 23 lines

```
typedef Point<int> P;
vector<array<int, 3>> manhattanMST(vector<P> ps) {
  vi id(sz(ps));
  iota(all(id), 0);
  vector<array<int, 3>> edges;
  rep(k,0,4) {
    sort(all(id), [&](int i, int j) {
         return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j});
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.
    y);
  }
  return edges;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h" bac5b0, 63 lines

```
typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

bool on_x(const P& a, const P& b) { return a.x < b.x; }
bool on_y(const P& a, const P& b) { return a.y < b.y; }

struct Node {
  P pt; // if this is a leaf, the single point in it
  T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
  Node *first = 0, *second = 0;

  T distance(const P& p) { // min squared distance to a
    point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
    return (P(x,y) - p).dist2();
  }

  Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
      x0 = min(x0, p.x); x1 = max(x1, p.x);
      y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
      // split on x if width >= height (not ideal...)
      sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
      // divide by taking half the array for each child (not
      // best performance with many duplicates in the middle
      )
      int half = sz(vp)/2;
      first = new Node({vp.begin(), vp.begin() + half});
      second = new Node({vp.begin() + half, vp.end()});
    }
  }
};
```

```
struct KDTree {
  Node* root;
  KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}

  pair<T, P> search(Node *node, const P& p) {
    if (!node->first) {
      // uncomment if we should not find the point itself:
      // if (p == node->pt) return {INF, P()};
      return make_pair((p - node->pt).dist2(), node->pt);
    }

    Node *f = node->first, *s = node->second;
    T bfirst = f->distance(p), bsec = s->distance(p);
    if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

    // search closest side first, other side if needed
    auto best = search(f, p);
    if (bsec < best.first)
      best = min(best, search(s, p));
    return best;
  }

  // find nearest point to a point, and its squared distance
  // (requires an arbitrary operator< for Point)
  pair<T, P> nearest(const P& p) {
    return search(root, p);
  }
};
```

Delaunay.h

Description: Computes the Delaunay triangulation of a set of points. Each circumcircle contains none of the input points.
Time: $\mathcal{O}(hull3d)$

14d818, 16 lines

```
using P = Point<ll>;
vector<array<P,3>> triHull(vector<P> p) {
  vector<P3> p3; vector<array<P,3>> res; for (auto &x:p) p3.
    pb(P3{x.x,x.y,x.dist2()});
  bool ok = 0; for (auto &t:p3) ok |= !coplanar(p3[0],p3[1],
    p3[2],t);
  if (!ok) { // all points concyclic
    sort(1+all(p),[&p](P a, P b) {
      return (a-p.front()).cross(b-p.front())>0; });
    rep(i,1,sz(p)-1) res.pb({p.front(),p[i],p[i+1]});
  } else {
    #define nor(z) P(p3[z].x,p3[z].y)
    for(auto &t:hull3dFast(p3))
      if (cross(p3[t[0]],p3[t[1]],p3[t[2]]).dot(P3{0,0,1}) <
      0)
        res.pb({nor(t[0]),nor(t[2]),nor(t[1])});
  }
  return res;
}
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.
Time: $\mathcal{O}(n \log n)$

"Point.h" eefdf5, 87 lines

```
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point

struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2;
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B >
    0;
}
Q makeEdge(P orig, P dest) {
```

```
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r
    ();
  r->p = orig; r->F() = dest;
  return r;
}
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
}
pair<Q,Q> rec(const vector<P>& s) {
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back())
     ;
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({sz(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  }
  return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
  sort(all(pts));  assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p)
    ; \
  q.push_back(c->r()); c = c->next(); } while (c != e); }
  ADD; pts.clear();
  while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
  return pts;
}
```

8.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards. 7af6b2, 7 lines

```
template<class V, class L>
auto signedPolyVolume(const V& p, const L& trilist) {
  using T = decltype(p[0].dot(p[0]));
  conditional_t<is_integral<T>::value, double, T> v = 0;
  for (auto i : trilist) v += p[i.a].cross(p[i.b]).dot(p[i.c
    ]);
  return v / 6;
}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double, long long or Fraction. With Fraction, unit/normal/rotate require conversion to floating-point points. 48f2bd, 33 lines

```
template<class T> struct Point3D {
  typedef Point3D P;
  typedef const P& R;
  T x, y, z;
  explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z)
    {}
  bool operator<(R p) const {
    return tie(x, y, z) < tie(p.x, p.y, p.z); }
  bool operator==(R p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z); }
  P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
  P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
  P operator*(T d) const { return P(x*d, y*d, z*d); }
  P operator/(T d) const { return P(x/d, y/d, z/d); }
  T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
  P cross(R p) const {
    return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
  }
  T dist2() const { return x*x + y*y + z*z; }
  double dist() const { return sqrt((double)dist2()); }
  //Azimuthal angle (longitude) to x-axis in interval [-pi,
    pi]
  double phi() const { return atan2((double)y, (double)x); }
  //Zenith angle (latitude) to the z-axis in interval [0, pi
    ]
  double theta() const {
    return atan2(sqrt((double)(x*x+y*y)), (double)z); }
  P unit() const { return *this/(T)dist(); } //makes dist()
    =1
  //returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); }
  //returns point rotated 'angle' radians ccw around axis
  P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u = axis.unit()
     ;
    return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
  }
};
```

3dHull-template-benq.h

Description: 3d hull helpers 481285, 36 lines

```
using T = double;
using P3 = Point3D<double>;
using vb = vector<bool>;

mt19937 rng;
P3 cross(const P3& a, const P3& b, const P3& c) {
  return (b-a).cross(c-a); }
P3 perp(const P3& a, const P3& b, const P3& c) {
  return cross(a,b,c).unit(); }
bool isMult(const P3& a, const P3& b) { // for long longs
  P3 c = a.cross(b);
  return (c.x== 0 && c.y==0 && c.z==0);
}
bool collinear(const P3& a, const P3& b, const P3& c) {
  return isMult(b-a,c-a); }
T DC(const P3&a,const P3&b,const P3&c,const P3&p) {
  return cross(a,b,c).dot(p-a); }
bool coplanar(const P3&a,const P3&b,const P3&c,const P3&p) {
  return DC(a,b,c,p) == 0; }
bool above(const P3&a,const P3&b,const P3&c,const P3&p) {
  return DC(a,b,c,p) > 0; } // is p strictly above plane
void prep(vector<P3>& p) { // rearrange points such that
  shuffle(all(p),rng); // first four are not coplanar
  int dim = 1;
  rep(i,1,sz(p))
    if (dim == 1) {
      if (p[0] != p[i]) swap(p[1],p[i]), ++dim;
    } else if (dim == 2) {
      if (!collinear(p[0],p[1],p[i]))
        swap(p[2],p[i]), ++dim;
    } else if (dim == 3) {
      if (!coplanar(p[0],p[1],p[2],p[i]))
        swap(p[3],p[i]), ++dim;
    }
  assert(dim == 4);
}
```

3dHull-slow-benq.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}\left(n^2\right)$ 0ba821, 27 lines

```
using F = array<int,3>; // face
vector<F> hull3d(vector<P3>& p) {
  // s.t. first four points form tetra
  prep(p); int N = sz(p); vector<F> hull; // triangle for
    each face
  auto ad = [&](int a, int b, int c) { hull.pb({a,b,c}); };
  // +new face to hull
  ad(0,1,2), ad(0,2,1); // initialize hull as first 3 points
  vector<vb> in(N,vb(N)); // is zero before each iteration
  rep(i,3,N) { // incremental construction
    vector<F> def, HULL; swap(hull,HULL);
    // HULL now contains old hull
    auto ins = [&](int a, int b, int c) {
      if (in[b][a]) in[b][a] = 0; // kill reverse face
      else in[a][b] = 1, ad(a,b,c);
    };
    for (auto &f:HULL) {
      if (above(p[f[0]],p[f[1]],p[f[2]],p[i]))
        rep(j,0,3) ins(f[j],f[(j+1)%3],i);
        // recalc all faces s.t. point is above face
      else def.pb(f);
    }
    for (auto &t:hull) if (in[t[0]][t[1]]) // edge exposed,
      in[t[0]][t[1]] = 0, def.pb(t); // add a new face
    swap(hull,def);
  }
  return hull;
}
```

3dHull-fast-benq.h

Description: Computes all faces of the 3-dimension hull of a point set.

Time: $\mathcal{O}\left(n \log n\right)$ c9ba16, 46 lines

```
using F = array<int,3>; // face
vector<F> hull3dFast(vector<P3>& p) {
  prep(p); int N = sz(p); vector<F> hull;
  vb active; // whether face is active
  vector<vi> rvis; // points visible from each face
  vector<array<pi,3>> other; // other face adjacent to each
    edge of face
  vector<vi> vis(N); // faces visible from each point
  auto ad = [&](int a, int b, int c) {
    hull.pb({a,b,c}); active.pb(1); rvis.emplace_back();
     other.emplace_back(); };
  auto ae = [&](int a, int b) { vis[b].pb(a), rvis[a].pb(b);
     };
  auto abv = [&](int a, int b) {
    F f=hull[a]; return above(p[f[0]],p[f[1]],p[f[2]],p[b])
     ;};
  auto edge = [&](pi e) -> pi {
    return {hull[e.first][e.second],hull[e.first][(e.second+
     1)%3]}; };
  auto glue = [&](pi a, pi b) { // link two faces by an edge
    pi x = edge(a); assert(edge(b) == mp(x.second,x.first));
    other[a.first][a.second] = b, other[b.first][b.second] =
      a;
  }; // ensure face 0 is removed when i=3
  ad(0,1,2), ad(0,2,1); if (abv(1,3)) swap(p[1],p[2]);
  rep(i,0,3) glue({0,i},{1,2-i});
  rep(i,3,N) ae(abv(1,i),i); // coplanar points go in rvis
    [0]
  vi label(N,-1);
  rep(i,3,N) { // incremental construction
    vi rem; for(auto &t:vis[i]) if (active[t]) active[t]=0,
     rem.pb(t);
    if (!sz(rem)) continue; // hull unchanged
    int st = -1;
    for(auto &r:rem) rep(j,0,3) {
      int o = other[r][j].first;
      if (active[o]) { // create new face!
```

```
        int a,b; tie(a,b) = edge({r,j}); ad(a,b,i); st = a;
        int cur = sz(rvis)-1; label[a] = cur;
        vi tmp; set_union(all(rvis[r]),all(rvis[o]),
                  back_inserter(tmp));
        // merge sorted vectors ignoring duplicates
        for(auto &x:tmp) if (abv(cur,x)) ae(cur,x);
        glue({cur,0},other[r][j]); // glue old w/ new face
      }
    }
    for (int x = st, y; ; x = y) { // glue new faces
     together
      int X = label[x]; glue({X,1},{label[y=hull[X][1]],2});
      if (y == st) break;
    }
  }
  vector<F> ans; rep(i,0,sz(hull)) if (active[i]) ans.pb(
     hull[i]);
  return ans;
}
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

611f07, 8 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(d/2);
}
```

Strings (9)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

291ecf, 26 lines

```
vi pi(const string& s) {
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}
vi match(const string& s, const string& pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
  return res;
}
vi match2(const string& s, const string& pat) { // only
     compute pi for pat
  vi p = pi(pat), res;
  int cp = 0;
  rep(i,1,sz(s)) {
    int g = cp;
    while (g && s[i] != pat[g]) g = p[g-1];
    cp = g + (s[i] == pat[g]);
    if (cp >= sz(pat)) res.push_back(i - sz(pat) + 1);
  }
  return res;
}
```

Zfunc.h

Description: z[x] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

ee09e2, 12 lines

```
vi Z(const string& S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i (i is right of middle, s[i]==s[i-1]), p[1][i] = longest odd (half rounded down).

Time: $\mathcal{O}(N)$

e7ad79, 13 lines

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`

Time: $\mathcal{O}(N)$

d07a42, 8 lines

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1);
     break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  }
  return a;
}
```

SuffixArray.h

Description: Builds suffix array for a string. `sa[i]` is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n+1$, and `sa[0] = n`. The `lcp` array contains longest common prefixes for neighbouring strings in the suffix array: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. The input string must not contain any zero bytes.

Time: $\mathcal{O}(n \log n)$

ef440a, 44 lines

```
struct SuffixArray {
  vi sa, lcp; // sa[0] is empty str, size is n+1, lcp[i] is
     of sa[i] and sa[i-1]
  SuffixArray(string& s, int lim=256) { // or basic_string<
     int>
    int n = sz(s) + 1, k = 0, a, b;
    vi x(all(s)+1), y(n), ws(max(n, lim)), rank(n);
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p
     ) {
      p = j, iota(all(y), n - j);
      rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      rep(i,0,n) ws[x[i]]++;
      rep(i,1,lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p
     ++;
    }
    rep(i,1,n) rank[sa[i]] = i;
    for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
      for (k && k--, j = sa[rank[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
  // total unique substrings = (n+1 C 2) - sum(lcp)
  // string search in O(m log n). returns [l,r] of matches
     in sa.
  pair<int,int> search(string &t) {
    int b=1,e=sz(sa)-1;
    while (b<e) {
      int m=(b+e)/2;
      if (s.compare(sa[m], sz(t), t) >= 0) e = m; // not
     strict
      else b = m+1;
    }
    if (s.compare(sa[b], sz(t), t) != 0) return mp(-1,-1);
    pair<int,int> ans{b,-1};
    b=1,e=sz(sa)-1;
    while (b<e) {
      int m=(b+e)/2;
      if (s.compare(sa[m], sz(t), t) > 0) e = m; // strict
      else b = m+1;
    }
    if (s.compare(sa[b], sz(t), t) != 0) e--;
    ans.second = e;
    return ans;
  }
};
```

SuffixAutomaton.h

Description: what it says

Usage: `just do it`

bfbd86, 39 lines

```
struct SA {
  vector<map<char, int> > adj;
  vi link, dis;
  int N;
  SA(): adj(1), link(1, -1), dis(1, 0), N(1) {}
  int new_node(int v=-1) {
    if(v == -1)
      adj.emplace_back(), link.emplace_back(), dis.
     emplace_back();
    else
      adj.push_back(adj[v]), link.push_back(link[v]), dis.
     push_back(dis[v]);
    return N++;
  }
  int go(int p, int c) {
    auto it = adj[p].find(c);
    if(dis[it->second] == dis[p] + 1)
      return it->second;
    else {
      int q = it->second, n = new_node(q);
      dis[n] = dis[p] + 1, link[q] = n;
      for(;p != -1 && (it = adj[p].find(c))->second == q;p =
      link[p])
        it->second = n;
      return n;
    }
  }
  int append(int p, char c) {
    auto it = adj[p].find(c);
    if(it != adj[p].end())
      return go(p, c);
    int n = new_node();
    dis[n] = dis[p] + 1;
    for(;p != -1 && adj[p].find(c) == adj[p].end();p = link[
     p])
      adj[p].insert({c, n});
    if(p == -1)
      link[n] = 0;
    else
      link[n] = go(p, c);
    return n;
  }
};
```

Hashing.h

Description: Self-explanatory methods for string hashing.

2d2a67, 44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod
     2^64).
// "typedef ull H;" instead if you think test data is random
     ,
// or work mod 10^9+7 if the Birthday paradox is not a
     problem.
```

```
typedef uint64_t ull;
struct H {
  ull x; H(ull x=0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) { auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64); }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11+3; // (order ~ 3e9; random also ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
    pw[0] = 1;
    rep(i,0,sz(str))
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};

vector<H> getHashes(string& str, int length) {
  if (sz(str) < length) return {};
  H h = 0, pw = 1;
  rep(i,0,length)
    h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  rep(i,length,sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i-length]);
  }
  return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
```

PalTree.h

Description: trie of right-half of all palindromic substrings link points to longest suffix-palindrome of node oc stores num occurences after resolveOc() ans[i][b] is min factorization of prefix i (1-indexed) with parity b

Time: $\mathcal{O}(n \log n)$ 0a57d3, 39 lines

```
struct PalTree {
    static const int ASZ = 26;
    struct node {
        array<int,ASZ> to = array<int,ASZ>();
        int len, link, oc = 0; // # occurrences of pal
        int slink = 0, diff = 0;
        array<int,2> seriesAns;
        node(int _len, int _link) : len(_len), link(_link) {}
    };
    string s = "@"; vector<array<int,2>> ans = {{0,INF}};
    vector<node> d = {{0,1},{-1,0}}; // dummy pals of len 0,-1
    int last = 1;
    int getLink(int v) {
        while (s[sz(s)-d[v].len-2] != s.back()) v = d[v].link;
        return v;
    }
    void updAns() { // serial path has O(log n) vertices
        ans.pb({INF,INF});
        for (int v = last; d[v].len > 0; v = d[v].slink) {
            d[v].seriesAns=ans[sz(s)-1-d[d[v].slink].len-d[v].diff];
            if (d[v].diff == d[d[v].link].diff)
                rep(i,0,2) ckmin(d[v].seriesAns[i], d[d[v].link].seriesAns[i]);
            // start of previous oc of link[v]=start of last oc of v
            rep(i,0,2) ckmin(ans.back()[i],d[v].seriesAns[i^1]+1);
        }
    }
    void addChar(char C) {
        s += C; int c = C-'a'; last = getLink(last);
        if (!d[last].to[c]) {
            d.emplace_back(d[last].len+2,d[getLink(d[last].link)].to[c]);
            d[last].to[c] = sz(d)-1;
            auto& z = d.back(); z.diff = z.len-d[z.link].len;
            z.slink = z.diff == d[z.link].diff ? d[z.link].slink : z.link;
        } // max suf with different dif
        last = d[last].to[c]; ++d[last].oc;
        updAns();
    }
    void resolveOc() { for (int i=sz(d)-1;i>=2;i--) d[d[i].link].oc += d[i].oc; }
};
```

AhoCorasickJacob.h

Description: Aho-Corasick with unfixed, trie of all strings Link stores node of longest suffix that is also in the trie Only works after adding all strings and calling build().

5cfbad, 44 lines

```
struct AhoCorasick {
    struct node {
        unordered_map<char,int> next; // can use array
        int link = -1;
        int output = 0; // adapt as necessary
        node() {}
    };
    vector<node> nodes{1};
    void add_string(const string& s) {
        int v = 0;
        for (char c:s) {
            if (nodes[v].next.find(c) == nodes[v].next.end()) {
                nodes[v].next[c] = sz(nodes);
                nodes.emplace_back();
            }
            v = nodes[v].next[c];
        }
        nodes[v].output = 1; // update output
    }
    int nx(int j, char c) {
        while (j != -1 && !nodes[j].next.count(c)) j = nodes[j].link;
        return (j == -1) ? 0 : nodes[j].next[c];
    }
    void build() {
        queue<int> q; q.push(0);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (auto p : nodes[v].next) {
                int x = p.second;
                nodes[x].link = nx(nodes[v].link, p.first);
                // update output if needed
                q.push(x);
            }
        }
    }
    int match(const string &text) {
        int n = sz(text), v = 0;
        for (int i = 0; i < n; i++) {
            v = nx(v, text[i]);
            if (nodes[v].output) return i;
        }
        return -1;
    }
};
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).

Time: $\mathcal{O}(\log N)$ edce47, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}

void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add `|| R.empty()`. Returns empty set on failure (or if G is empty).

Time: $\mathcal{O}(N \log N)$ 9e9d8d, 19 lines

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
  vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
  T cur = G.first;
  int at = 0;
  while (cur < G.second) { // (A)
    pair<T, int> mx = make_pair(cur, -1);
    while (at < sz(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

Usage: `constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});`

Time: $\mathcal{O}\left(k \log \frac{n}{k}\right)$ 753a4c, 19 lines

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
  if (p == q) return;
  if (from == to) {
    g(i, to, p);
    i = to; p = q;
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid+1, to, f, g, i, p, q);
  }
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
  if (to <= from) return;
  int i = from; auto p = f(i), q = f(to-1);
  rec(from, to-1, f, g, i, p, q);
  g(i, to, q);
}
```

10.2 Misc. algorithms

Dates.h

Description: Dates

74f735, 42 lines

```
// Routines for performing computations on dates.  In these routines,
```

```
// months are expressed as integers from 1 to 12, days are expressed
// as integers from 1 to 31, and years are expressed as 4-digit
// integers.
string dayOfWeek[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
// converts Gregorian date to integer (Julian day number)
int dateToInt (int m, int d, int y){
  return
    1461 * (y + 4800 + (m - 14) / 12) / 4 +
    367 * (m - 2 - (m - 14) / 12 * 12) / 12 -
    3 * ((y + 4900 + (m - 14) / 12) / 100) / 4 +
    d - 32075;
}
// converts integer (Julian day number) to Gregorian date: month/day/year
void intToDate (int jd, int &m, int &d, int &y){
  int x, n, i, j;
  x = jd + 68569;
  n = 4 * x / 146097;
  x -= (146097 * n + 3) / 4;
  i = (4000 * (x + 1)) / 1461001;
  x -= 1461 * i / 4 - 31;
  j = 80 * x / 2447;
  d = x - 2447 * j / 80;
  x = j / 11;
  m = j + 2 - 12 * x;
  y = 100 * (n - 49) + i + x;
}
// converts integer (Julian day number) to day of week
string intToDay (int jd){ return dayOfWeek[jd % 7]; }
/*int main (int argc, char **argv){
  int jd = dateToInt (3, 24, 2004);
  int m, d, y;
  intToDate (jd, m, d, y);
  string day = intToDay (jd);
  // expected output:
  //    2453089
  //    3/24/2004
  //    Wed
  cout << jd << endl
    << m << "/" << d << "/" << y << endl
    << day << endl;
}*/
```

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).
Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`
Time: $\mathcal{O}(\log(b-a))$ 133dcd, 11 lines

```
template<class F>
int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = a + (b - a) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  }
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

LIS.h

Description: Compute indices for the longest increasing subsequence.
Time: $\mathcal{O}(N \log N)$ 2932a0, 17 lines

```
template<class I> vi lis(const vector<I>& S) {
  if (S.empty()) return {};
  vi prev(sz(S));
  typedef pair<I, int> p;
  vector<p> res;
  rep(i,0,sz(S)) {
    // change 0 -> i for longest non-decreasing subsequence
    auto it = lower_bound(all(res), p{S[i], 0});
    if (it == res.end()) res.emplace_back(), it = res.end()-1;
    *it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it-1)->second;
  }
  int L = sz(res), cur = res.back().second;
  vi ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$ b20ccc, 16 lines

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
  while (b < sz(w) && a + w[b] <= t) a += w[b++];
  if (b == sz(w)) return a;
  int m = *max_element(all(w));
  vi u, v(2*m, -1);
  v[a+m-t] = b;
  rep(i,b,sz(w)) {
    u = v;
    rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
    for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
      v[x-w[j]] = max(v[x-w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
}
```

10.3 128-bit numbers and I/O

10.3.1 Types, precision, and compiler support

`__int128` (also `__int128_t`) is a signed integer in $[-2^{127}, 2^{127}-1]$; `unsigned __int128` (`__uint128_t`) is in $[0, 2^{128}-1]$. Signed overflow is undefined, even with 128 bits. Widen *before* arithmetic: `__int128(a)*b`, not `__int128(a*b)`. For differences, widen before subtraction too.

Typical x86-64 GNU C++ floating-point precision:

Type	Significand bits	Decimal digits (approx.)
`double`	53	16
`long double`	64	19
`__float128`	113	34

`long double` is platform-dependent: 16 bytes of storage can mean 80-bit extended precision plus padding, not quadruple precision. Check `numeric_limits<long double>::digits`, or `LDBL_MANT_DIG` from `<cfloat>`; `FLT128_MANT_DIG` from `<quadmath.h>` is 113. `using LD = __float128;` makes `LD` quadruple precision regardless of the alias's name.

Both 128-bit types are compiler extensions, not C++17 additions. They work with `g++ -std=c++17` on supported targets; a newer language standard does not change `long double` precision. C++23 separately introduced the optional `std::float128_t` in `<stdfloat>`. Availability depends on the compiler, standard library, and target.

10.3.2 Input and output

For the ordinary types, use matching formats:

Type	scanf	printf
`long long`	`%lld`	`%lld`
`unsigned long long`	`%llu`	`%llu`
`double`	`%lf`	`%f`
`long double`	`%Lf`	`%Lf`

These ordinary types support `cin` and `cout` directly. In GNU C++17, the 128-bit extension types have no direct standard stream overloads. Never pass `__int128` to `%lld` or `__float128` to `%Lf`: mismatched variadic arguments are invalid.

Integer conversion

There is no standard `scanf` or `printf` format for `__int128`. If input values fit in `long long`, read that type and then widen. Otherwise read a string and parse it. The following helpers handle the full signed range, including the minimum value. `parse128` accepts an optional sign and decimal digits; on invalid input or overflow it returns false without changing the output.

Int128IO.h

Description: Decimal conversion for the full signed 128-bit range. parse128 accepts an optional sign followed by digits. It returns false on invalid input or overflow, leaving out unchanged. str128 handles the minimum value without signed negation overflow.
Time: $\mathcal{O}(|s|)$ parsing, $\mathcal{O}(\log|x|)$ formatting. 25fd8d, 28 lines

```
bool parse128(const string& s, __int128_t& out) {
  if (s.empty()) return false;
  bool neg = s[0] == '-';
  size_t i = neg || s[0] == '+';
  if (i == s.size()) return false;
  using U = __uint128_t;
  U n = 0, limit = (U(1) << 127) - !neg;
  for (; i < s.size(); ++i) {
    unsigned d = unsigned(s[i] - '0');
    if (d > 9 || n > (limit-d)/10) return false;
    n = n*10 + d;
  }
  out = neg && n ? -__int128_t(n-1)-1 : __int128_t(n);
  return true;
}

string str128(__int128_t x) {
  using U = __uint128_t;
  U n = x < 0 ? -U(x) : U(x);
  string s;
  do {
    s += char('0' + n%10);
    n /= 10;
  } while (n);
  if (x < 0) s += '-';
  reverse(all(s));
  return s;
}
```

```
string token;
__int128_t n;
if (cin >> token && parse128(token, n))
    cout << str128(n) << '\n';
```

Quadruple-precision conversion

Use `strtoflt128` for input and `quadmath_snprintf` for output. Include `<quadmath.h>` and link with `-lquadmath`:

```
g++ -std=c++17 -O2 solution.cpp -lquadmath
```

This complete program assumes a valid floating-point input token:

```
#include <quadmath.h>
#include <iostream>
#include <string>
using namespace std;

int main() {
    string token;
    if (!(cin >> token)) return 0;
    __float128 x = strtoflt128(token.c_str(), nullptr);
    char buf[128];
    int n = quadmath_snprintf(buf, sizeof(buf), "%.36Qg", x);
    if (n < 0 || n >= int(sizeof(buf))) return 1;
    cout << buf << '\n';
```

```
}
```

`%.36Qg` prints 36 significant digits, sufficient for binary128 round trips. `%.15Qf` prints 15 digits after the decimal point; `%Qe` uses scientific notation. The format must contain exactly one conversion, with no surrounding text (not even a newline). Print text and newlines separately. A fixed-point result can require thousands of digits; check the returned length and increase the buffer if needed.

Ordinary `printf("%Qf", x)` only works with some C-library hooks; do not rely on it. There is no portable matching `scanf` conversion. For unchecked input, use `strtoflt128`'s end pointer to check that the whole token was read, and inspect `errno` for range errors.

10.3.3 Avoiding accidental precision loss

- `__float128 x = 0.1;` first rounds the literal to `double`. Prefer `__float128(1)/10` or `strtoflt128("0.1", nullptr)`. GCC's `Q` literal suffix may need `-fext-numeric-literals` in strict C++ mode; the examples above do not need it.
- Basic 128-bit arithmetic does not require `-lquadmath`. Quadmath conversions and functions such as `sqrtq` and `fabsq` do. Ordinary `cmath` overloads are not a substitute.
- `cout << (long double)x` is an approximate output shortcut, not full-precision binary128 output. Cast only at the end, and only if the problem's tolerance permits the loss. More printed digits cannot recover precision already lost in a conversion or calculation.
- For integer-coordinate geometry, keep predicates in sufficiently wide integer arithmetic and convert only when a division is required. Approximate intersection coordinates can lose accuracy again when used to calculate a thin triangle's area. More precision helps cancellation; it does not make floating-point predicates exact. An algebraic area formula may avoid constructing rounded coordinates altogether.

Sources: GCC's manuals for 128-bit integers, floating types, quadmath I/O, and the C++23 fixed-width floating types.

10.4 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \le f(a,d)$ and $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ for all $a \le b \le c \le d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

Time: $\mathcal{O}\left(N^2\right)$

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i)\le k<hi(i)}(f(i,k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R-1$.

Time: $\mathcal{O}\left((N + (hi-lo))\log N\right)$ d38d2b, 18 lines

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
  }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

10.5 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.6 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.6.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)];` computes all sums of subsets.

10.6.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$. 751a02, 8 lines

```
typedef unsigned long long ull;
struct FastMod {
  ull b, m;
  FastMod(ull b) : b(b), m(-1ULL / b) {}
  ull reduce(ull a) { // a % b + (0 or b)
    return a - (ull)((__uint128_t(m) * a) >> 64) * b;
  }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.

Usage: `./a.out < input.txt`

Time: About 5x as fast as cin/scanf. 8b1a63, 18 lines

```
inline char gc() { // like getchar()
  static char buf[1 << 16];
  static size_t bc, be;
  if (bc >= be) {
    buf[0] = 0, bc = 0;
    be = fread(buf, 1, sizeof(buf), stdin);
  }
  return buf[bc++]; // returns 0 on EOF
}

int readInt() {
  int a, c;
  while ((a = gc()) < 40);
  if (a == '-') return -readInt();
  a -= 48;
  while ((c = gc()) >= 48) a = a * 10 + (c - 48);
  return a;
}
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation. 745db2, 8 lines

```
// Either globally or in a single class:
static char buf[450 << 20];
void* operator new(size_t s) {
  static size_t i = sizeof buf;
  assert(s < i);
  return (void*)&buf[i -= s];
}
void operator delete(void*) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

"BumpAllocator.h" 2dd6c9, 10 lines

```
template<class T> struct ptr {
  unsigned ind;
  ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
    assert(ind < sizeof buf);
  }
  T& operator*() const { return *(T*)(buf + ind); }
  T* operator->() const { return &**this; }
  T& operator[](int a) const { return (&**this)[a]; }
  explicit operator bool() const { return ind; }
};
```

Techniques (A)

techniques.txt 159 lines

```
Recursion
Divide and conquer
  Finding interesting points in N log N
Algorithm analysis
  Master theorem
  Amortized time complexity
Greedy algorithm
  Scheduling
  Max contiguous subvector sum
  Invariants
  Huffman encoding
Graph theory
  Dynamic graphs (extra book-keeping)
  Breadth first search
  Depth first search
  * Normal trees / DFS trees
  Dijkstra's algorithm
  MST: Prim's algorithm
  Bellman-Ford
  Konig's theorem and vertex cover
  Min-cost max flow
  Lovasz toggle
  Matrix tree theorem
  Maximal matching, general graphs
  Hopcroft-Karp
  Hall's marriage theorem
  Graphical sequences
  Floyd-Warshall
  Euler cycles
  Flow networks
  * Augmenting paths
  * Edmonds-Karp
  Bipartite matching
  Min. path cover
  Topological sorting
  Strongly connected components
  2-SAT
  Cut vertices, cut-edges and biconnected components
  Edge coloring
  * Trees
  Vertex coloring
  * Bipartite graphs (=> trees)
  * 3^n (special case of set cover)
  Diameter and centroid
  K'th shortest path
  Shortest cycle
Dynamic programming
  Knapsack
  Coin change
  Longest common subsequence
  Longest increasing subsequence
  Number of paths in a dag
  Shortest path in a dag
  Dynprog over intervals
  Dynprog over subsets
  Dynprog over probabilities
  Dynprog over trees
  3^n set cover
  Divide and conquer
  Knuth optimization
  Convex hull optimizations
  RMQ (sparse table a.k.a 2^k-jumps)
  Bitonic cycle
  Log partitioning (loop over most restricted)
Combinatorics
  Computation of binomial coefficients
  Pigeon-hole principle
  Inclusion/exclusion
  Catalan number
  Pick's theorem
Number theory
  Integer parts
  Divisibility
  Euclidean algorithm
  Modular arithmetic
  * Modular multiplication
  * Modular inverses
  * Modular exponentiation by squaring
  Chinese remainder theorem
  Fermat's little theorem
  Euler's theorem
  Phi function
  Frobenius number
  Quadratic reciprocity
  Pollard-Rho
  Miller-Rabin
  Hensel lifting
  Vieta root jumping
Game theory
  Combinatorial games
  Game trees
  Mini-max
  Nim
  Games on graphs
  Games on graphs with loops
  Grundy numbers
  Bipartite games without repetition
  General games without repetition
  Alpha-beta pruning
Probability theory
Optimization
  Binary search
  Ternary search
  Unimodality and convex functions
  Binary search on derivative
Numerical methods
  Numeric integration
  Newton's method
  Root-finding with binary/ternary search
  Golden section search
Matrices
  Gaussian elimination
  Exponentiation by squaring
Sorting
  Radix sort
Geometry
  Coordinates and vectors
  * Cross product
  * Scalar product
  Convex hull
  Polygon cut
  Closest pair
  Coordinate-compression
  Quadtrees
  KD-trees
  All segment-segment intersection
Sweeping
  Discretization (convert to events and sweep)
  Angle sweeping
  Line sweeping
  Discrete second derivatives
Strings
  Longest common substring
  Palindrome subsequences
  Knuth-Morris-Pratt
  Tries
  Rolling polynomial hashes
  Suffix array
  Suffix tree
  Aho-Corasick
  Manacher's algorithm
  Letter position lists
Combinatorial search
  Meet in the middle
  Brute-force with pruning
  Best-first (A*)
  Bidirectional search
  Iterative deepening DFS / A*
Data structures
  LCA (2^k-jumps in trees in general)
  Pull/push-technique on trees
  Heavy-light decomposition
  Centroid decomposition
  Lazy propagation
  Self-balancing trees
  Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
  Monotone queues / monotone stacks / sliding queues
  Sliding queue using 2 stacks
  Persistent segment tree
```